Berliner Hochschule für Technik

Fachbereich VI – Informatik und Medien

Masterarbeit

von

Khaled Rafei

Matrikelnummer: 930677

zur Erlangung
des akademischen Grades

Master of Engineering (M. Eng.)

im Studiengang
Technische Informatik

Thema:

Dateneffiziente Personalisierung für webcam-basiertes Eye-Tracking mithilfe vortrainierter Bildverarbeitungsmodelle

Betreuer:	Prof. Dr. Meysam Shamsi
Erstgutachter:	Prof. Dr. Kristian Hildebrand
Zweitgutachter:	Prof. Dr. Robert Strzebkowski
Eingereicht:	05. 11. 2026

Kurzfassung

Webcam-basiertes Eye-Tracking ist kostengünstig und auf fast jedem Laptop verfügbar. Mit allgemeinen Modellen ist es aber oft ungenau, weil sich Personen, Kameras und Umgebungen stark unterscheiden. Diese Arbeit untersucht, wie sich ein vortrainiertes Bildverarbeitungsmodell mit wenigen Kalibrierungsdaten an eine einzelne Person anpassen lässt.

Als Merkmalsextraktor dient ein auf ImageNet vortrainiertes ResNet-18, das eingefroren bleibt. Trainiert wird nur ein kleiner Regressionskopf: zuerst auf dem Datensatz GazeCapture (Basismodell) und danach mit eigenen Webcam-Aufnahmen aus 18 Sitzungen einer Person (Personalisierung). Als Fehlermaß dient der Diagonalfehler, also der Fehler in Prozent der Bildschirmdiagonale.

Auf den eigenen Aufnahmen ist das Basismodell nicht besser als eine konstante Schätzung der Bildschirmmitte (30,6 % gegenüber 30,3 %). Die Personalisierung senkt den Fehler auf 19,8 %, also um etwa ein Drittel. Ein Trainingslauf dauert auf einer CPU nur wenige Sekunden. Bei einer realistischen, zeitlichen Aufteilung der Daten ist der Gewinn kleiner (etwa 12 % gegenüber der konstanten Schätzung). In Sitzungen mit guten Aufnahmebedingungen bleibt er jedoch deutlich und überträgt sich auch auf eine neue Aufnahme. Hintergrund und Beleuchtung haben einen größeren Einfluss als die Gestaltung der Kalibrierung. Das Entfernen falsch gelabelter Bilder nach Zielsprüngen verbessert das Ergebnis nur wenig.

Die Ergebnisse zeigen: Eine dateneffiziente Personalisierung ist mit einfachen Mitteln möglich. Ihr Nutzen muss aber mit einer realistischen Aufteilung der Daten bewertet werden.

Schlagwörter: Eye-Tracking, Blickschätzung, Webcam, Personalisierung, Transfer Learning, ResNet-18, GazeCapture

Inhaltsverzeichnis

Kurzfassung I

Abbildungsverzeichnis V

Tabellenverzeichnis VI

Abkürzungsverzeichnis VII

1. Einleitung 1
- 1.1. Motivation . . . 1
- 1.2. Problemstellung und Ziel . . . 1
- 1.3. Forschungsfragen . . . 2
- 1.4. Beitrag der Arbeit . . . 2
- 1.5. Aufbau der Arbeit . . . 3

2. Stand der Technik 4
- 2.1. Grundlagen der erscheinungsbasierten Blickschätzung . . . 4
  - 2.1.1. Klassische Verfahren . . . 4
  - 2.1.2. Erste Deep-Learning-Verfahren . . . 4
  - 2.1.3. Einteilung der Verfahren . . . 5
- 2.2. Webcam-basiertes Eye-Tracking: Alltag und Labor . . . 6
- 2.3. Große Datensätze zur Blickschätzung . . . 7
- 2.4. Netzarchitekturen für die Blickregression . . . 8
  - 2.4.1. Convolutional Neural Networks . . . 8
  - 2.4.2. Vision Transformer . . . 8
  - 2.4.3. Vortrainierte Foundation-Modelle . . . 9
- 2.5. Vortrainierte Modelle für die Blickschätzung . . . 9
  - 2.5.1. Eingefrorener Backbone oder vollständiges Fine-Tuning . . . 9
  - 2.5.2. Großes MAE-Vortraining: UniGaze . . . 10
  - 2.5.3. Bild-Text-Modelle: CLIP-Gaze und GazeCLIP . . . 11
- 2.6. Dateneffiziente Personalisierung . . . 13
  - 2.6.1. Kalibrierverfahren . . . 14
  - 2.6.2. Few-Shot-Learning und Meta-Learning: FAZE . . . 15
  - 2.6.3. Parametereffizientes Fine-Tuning (PEFT) . . . 17

2.7. Domänenwechsel, Generalisierung und Evaluationsprotokolle 17
2.7.1. Evaluation mit unbekannten Personen 17
2.7.2. Domänenadaption und Domänengeneralisierung 18
2.7.3. Blickschätzung für mehrere Personen 19
2.7.4. Unterschiede bei Geräten und Umgebung 19
2.8. Anpassung zur Testzeit 20
2.8.1. TTP-GE: selbstüberwachte und schwach überwachte Anpassung . 20
2.8.2. Weitere labelsparende Verfahren 20
2.9. Datenqualität, falsche Labels und zeitlicher Ablauf 21
2.9.1. Datenqualität im Eye-Tracking 22
2.9.2. Reaktionszeit und Sakkaden 22
2.9.3. Filtern schon bei der Aufnahme 22
2.9.4. Filtern nach der Aufnahme und robuste Lernverfahren 23
2.10. Vergleich und kritische Diskussion 24
2.11. Forschungslücken und Bezug zu dieser Arbeit 26
2.11.1. Forschungslücken 26
2.11.2. Was diese Arbeit beiträgt 26

3. Methodik **28**
3.1. Datensätze 28
3.2. Aufnahme der Personalisierungsdaten 28
3.3. Vorverarbeitung 29
3.4. Modell und Training 29
3.5. Auswertung 30
3.6. Implementierung 30

4. Experimente und Ergebnisse **32**
4.1. Versuchsaufbau 32
4.1.1. Modell 32
4.1.2. Daten und Splits 32
4.1.3. Fehlermaß 33
4.1.4. Ablauf der Experimente 33
4.1.5. Normierung der Zielwerte 34
4.1.6. Baselines 35
4.2. Trainingseinstellungen 37
4.2.1. Batchgröße 37
4.2.2. Lernrate 37
4.2.3. Trainierbare Schichten 39
4.2.4. Aktivierung am Ausgang 40
4.2.5. Datenmenge 41
4.2.6. Art des Splits 41

4.3. Räumliche Fehleranalyse und Verlustfunktionen . . . 41
4.3.1. Einteilung des Bildschirms . . . 42
4.3.2. Fehler-Heatmaps . . . 42
4.3.3. Verlustfunktionen . . . 43
4.3.4. Ergebnisse des Verlustvergleichs . . . 44
4.4. Personalisierungssitzungen . . . 46
4.4.1. Aufbau . . . 46
4.4.2. Ergebnisse aller Sitzungen (E1) . . . 47
4.4.3. Trainingszeit einer Sitzung . . . 50
4.4.4. Bereinigung der Labels (Gen 4, E1b) . . . 50
4.4.5. Aufnahmebedingungen (Gen 4) . . . 51
4.5. Strategien zum Aufteilen der Daten (E2) . . . 52
4.6. Gestaltung der Kalibriersitzung . . . 54
4.6.1. Abdeckung des Bildschirms und Übergangsbilder . . . 54
4.6.2. Ergebnis der Personalisierung (E3) . . . 55
4.7. Diskussion der Ergebnisse . . . 57
4.8. Zusammenfassung der Ergebnisse . . . 58

5. Diskussion **60**
5.1. Antworten auf die Forschungsfragen . . . 60
5.2. Einordnung in den Stand der Technik . . . 61
5.3. Grenzen der Arbeit . . . 61

6. Fazit und Ausblick **63**
6.1. Fazit . . . 63
6.2. Ausblick . . . 63

Literaturverzeichnis **65**

A. Anhang **70**
A.1. Befehle zum Wiederholen der Experimente . . . 70

Eidesstattliche Erklärung **72**

Abbildungsverzeichnis

4.1. Rohe und normierte Blickziele im Vergleich. 36
4.2. Testfehler einzelner Läufe mit vier Lernraten 38
4.3. Räumlicher Fehler des Standardmodells 43
4.4. Testfehler der 18 Sitzungen nach der Personalisierung mit beiden Basismodellen, verglichen mit der Konstant-Baseline und dem unveränderten Basismodell (`norm_subject`) auf denselben Testbildern (Daten aus Tabelle 4.13). 47

Tabellenverzeichnis

2.1. Überblick über ausgewählte Datensätze zur Blickschätzung. 8
2.2. Vergleich von UniGaze und dieser Arbeit. 12
2.3. Vergleich von CLIP-Gaze, GazeCLIP und dieser Arbeit. 14
2.4. Vergleich von FAZE und dieser Arbeit. 16
2.5. Vergleich von TTP-GE und dieser Arbeit. 21
2.6. Vergleich von nachträglicher Filterung (DMRF) und Phasenprotokoll. . . 24
2.7. Vergleich der wichtigsten Arbeiten 25

4.1. Standardeinstellung von `exp.py` 34
4.2. Baseline-Fehler E_{diag} in % 36
4.3. Batchgröße 37
4.4. Lernrate 38
4.5. Trainierbare Parameter der drei Varianten (ResNet-18 mit Regressionskopf, insgesamt 11 603 650 Parameter). 39
4.6. Trainierbare Schichten 39
4.7. Aktivierung am Ausgang 40
4.8. Datenmenge (Training/Test) 41
4.9. Art des Splits 41
4.10. Vergleich von L1- und sensitivem L1-Verlust 44
4.11. Zusatzexperiment: räumlicher Fehler von vier Verlustfunktionen 45
4.12. Aufnahmebedingungen der 18 Sitzungen 47
4.13. Personalisierung der 18 Sitzungen mit zwei Basismodellen 48
4.14. Bereinigung der Labels in Gen 4 51
4.15. Strategien zum Aufteilen der Daten 53
4.16. Abdeckung und Übergangsbilder pro Gestaltung 54
4.17. Testfehler pro Gestaltung mit zufälligem und sequenziellem Split 55
4.18. Testfehler pro Bildschirmregion 55
4.19. Zusammenfassung der Ergebnisse und der gewählten Einstellung 59

Abkürzungsverzeichnis

Abkürzung	Bedeutung
AdamW	Adam-Optimierer mit entkoppeltem Weight Decay
CLIP	Contrastive Language-Image Pre-training
CNN	Convolutional Neural Network (faltendes neuronales Netz)
CPU	Central Processing Unit (Hauptprozessor)
DA	Domänenadaption
DG	Domänengeneralisierung
DINOv2	Self-Distillation with No Labels, Version 2
FC	Fully Connected (vollständig verbundene Schicht)
fps	Frames per Second (Bilder pro Sekunde)
KI	Konfidenzintervall
L1	Verlust aus dem mittleren absoluten Fehler
LoRA	Low-Rank Adaptation
MAE	Masked Autoencoder
MAML	Model-Agnostic Meta-Learning
MLP	Multi-Layer Perceptron (mehrschichtiges Perzeptron)
PEFT	Parameter-Efficient Fine-Tuning (parametereffizientes Fine-Tuning)
PoR	Point of Regard (Blickpunkt auf dem Bildschirm)
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error (Wurzel des mittleren quadratischen Fehlers)
SD	Standardabweichung
SVR	Support-Vector-Regression
TTA	Test-Time Adaptation (Adaption zur Testzeit)
ViT	Vision Transformer
Formelzeichen	
E_{diag}	Diagonalfehler: euklidischer RMSE in Prozent der Bildschirmdiagonale

1. Einleitung

1.1. Motivation

Eye-Tracking misst, wohin ein Mensch schaut. Diese Information ist für viele Anwendungen nützlich: für die Bedienung von Computern ohne Hände, zum Beispiel für Menschen mit körperlichen Einschränkungen, für Studien zur Benutzerfreundlichkeit von Webseiten, für Lernplattformen oder für die Forschung in Psychologie und Medizin. Professionelle Eye-Tracker arbeiten mit Infrarotlicht und speziellen Kameras. Sie sind sehr genau, aber teuer und nicht überall verfügbar.

Eine normale Webcam ist dagegen in fast jedem Laptop eingebaut. Wenn man den Blick mit einer Webcam schätzen könnte, wäre Eye-Tracking für sehr viele Menschen ohne zusätzliche Hardware möglich. Das ist aber schwierig. Die Bilder haben eine niedrige Auflösung, und die Augen sind darin nur wenige Pixel groß. Das Licht ändert sich, der Kopf bewegt sich, viele Menschen tragen eine Brille, und jede Person hat eine andere Augen- und Gesichtsform. Ein allgemeines Modell, das auf vielen Personen trainiert wurde, passt deshalb oft schlecht zu einer neuen Person.

1.2. Problemstellung und Ziel

Ein Ausweg ist die *Personalisierung*: Man nimmt ein vortrainiertes Modell und passt es mit wenigen Daten an eine Person an. Die Person schaut dazu kurz auf einen Punkt, der sich über den Bildschirm bewegt (Kalibrierung). Die Kalibrierung soll aber kurz sein, denn niemand möchte vor jeder Nutzung mehrere Minuten lang kalibrieren. Die Personalisierung muss also *dateneffizient* sein.

Das Ziel dieser Arbeit ist es, zu untersuchen, wie gut ein vortrainiertes Bildverarbeitungsmodell mit einer kurzen Webcam-Kalibrierung an eine Person angepasst werden kann. Als Modell dient ein auf ImageNet vortrainiertes ResNet-18. Sein Backbone bleibt eingefroren, und nur ein kleiner Regressionskopf wird trainiert. Der Kopf wird zuerst auf dem Datensatz GazeCapture trainiert und danach mit eigenen Aufnahmen personalisiert.

1.3. Forschungsfragen

Die Arbeit beantwortet vier Forschungsfragen (FF):

FF1 Wie stark senkt eine kurze Personalisierung den Fehler gegenüber einem allgemeinen Modell und gegenüber einer einfachen Schätzung, die immer denselben Punkt ausgibt?

FF2 Welche Trainingseinstellung eignet sich für den Regressionskopf (trainierbare Schichten, Aktivierung, Verlustfunktion, Lernrate, Batchgröße)?

FF3 Wie beeinflussen die Aufteilung der Daten (zufällig, zeitlich, zwei Videos) und die Gestaltung der Kalibrierung (Bewegung des Ziels, Entfernen von Übergangsbildern) das Ergebnis?

FF4 Welchen Einfluss haben die Aufnahmebedingungen (Licht, Hintergrund, Brille, Dauer, Betriebssystem)?

1.4. Beitrag der Arbeit

Die Arbeit liefert folgende Beiträge:

- **Eigene Aufnahmen:** 27 Kalibriersitzungen mit einer Webcam, davon 18 ausgewertet. Die Sitzungen unterscheiden sich in der Bewegung des Ziels, im Licht, im Hintergrund, in der Brille, in der Dauer und im Betriebssystem. Die neueste Generation speichert für jedes Bild die Phase des Ziels (`WAIT` oder `MOVE`).
- **Reproduzierbare Pipeline:** feste Seeds, eingefrorener Backbone, gespeicherte Merkmale (Merkmals-Cache) und gespeicherte Vorhersagen. Ein Personalisierungslauf dauert damit etwa zwei Sekunden auf einer CPU.
- **Korrigiertes Fehlermaß:** Ein Fehler im Auswertungscode wurde gefunden und behoben. Baselines werden immer auf denselben Daten wie das Modell berechnet.
- **Systematische Studie:** Hyperparameter mit fünf Wiederholungen und Konfidenzintervallen, räumliche Analyse des Fehlers sowie ein Vergleich von Datenaufteilungen und Kalibriergestaltungen.
- **Web-Demo:** Eine Demo im Browser zeigt die Personalisierung live: Kalibrierung, Blickverfolgung und Auswertung (Kapitel 6).

1.5. Aufbau der Arbeit

Kapitel 2 beschreibt den Stand der Technik. Kapitel 3 erklärt Daten, Modell und Vorgehen. Kapitel 4 zeigt die Experimente und Ergebnisse. Kapitel 5 beantwortet die Forschungsfragen und nennt die Grenzen der Arbeit. Kapitel 6 fasst die Arbeit zusammen und gibt einen Ausblick.

2. Stand der Technik

Dieses Kapitel gibt einen Überblick über die Blickschätzung mit normalen RGB-Kameras. Zuerst erkläre ich die Grundlagen (Abschnitt 2.1) und die besonderen Probleme von Webcams (Abschnitt 2.2). Danach stelle ich wichtige Datensätze (Abschnitt 2.3), Netzarchitekturen (Abschnitt 2.4) und vortrainierte Modelle (Abschnitt 2.5) vor. Anschließend geht es um Personalisierung, Domänenwechsel, Anpassung zur Testzeit und Datenqualität. Am Ende leite ich die Forschungslücken ab, die diese Arbeit bearbeitet.

2.1. Grundlagen der erscheinungsbasierten Blickschätzung

Eye-Tracking bestimmt aus Bildern der Augen oder des Gesichts, wohin eine Person schaut. Es gibt zwei Arten von Ausgaben: die dreidimensionale Blickrichtung (meist zwei Winkel, Pitch und Yaw) oder den zweidimensionalen *Point of Regard* (PoR). Der PoR ist der Punkt auf dem Bildschirm, auf den die Person schaut. Diese Arbeit schätzt den 2D-PoR aus Bildern einer normalen Webcam.

2.1.1. Klassische Verfahren

Erscheinungsbasierte Verfahren lernen direkt eine Abbildung vom Bild zum Blickziel. Sie brauchen kein geometrisches Modell des Auges. Frühe Verfahren haben von Hand entworfene Bildmerkmale verwendet, zum Beispiel Local Binary Patterns (LBP), Histograms of Oriented Gradients (HoG) oder SIFT. Diese Merkmale wurden mit einfachen Regressoren kombiniert, etwa mit linearer Regression, Random Forests oder Support-Vector-Regression (SVR). Solche Verfahren rechnen schnell. Bei Kopfbewegungen und wechselndem Licht werden sie aber deutlich ungenauer, weil die Merkmale diese Änderungen nicht gut von der Blickinformation trennen.

2.1.2. Erste Deep-Learning-Verfahren

Mit Convolutional Neural Networks (CNNs) kann man Gesichts- und Augenbilder direkt auf Bildschirmpositionen abbilden. Krafka et al. [1] haben mit *iTracker* eines der ersten CNNs für mobiles Eye-Tracking ohne zusätzliche Hardware vorgestellt. Das Modell verarbeitet vier Eingaben in getrennten Zweigen: das linke Auge, das rechte Auge, das Gesicht

und ein binäres *Face Grid* (wo das Gesicht im Bild liegt). Danach werden die Merkmale zusammengeführt. Ohne Kalibrierung liegt der Fehler bei 1,71 cm auf Smartphones und bei 2,53 cm auf Tablets. Für die Personalisierung trainieren die Autoren eine SVR pro Nutzer auf den Merkmalen der Schicht `fc1`. Dafür nutzen sie Aufnahmen von bis zu 13 festen Kalibrierpunkten. Mit allen 13 Punkten sinkt der Fehler auf 1,34 cm bzw. 2,12 cm. Mit sehr wenigen Punkten kann sich die SVR aber zu stark an die Daten anpassen (Überanpassung). Das Modell läuft in Echtzeit auf mobilen Geräten.

Ein wichtiger Beitrag dieser Arbeit ist der Datensatz GazeCapture (Abschnitt 2.3). Er wurde per Crowdsourcing auf Smartphones und Tablets unter Alltagsbedingungen gesammelt. Er dient bis heute als Benchmark und als Quelle für das Vortraining.

Bedeutung für diese Arbeit. iTracker hat drei Ideen eingeführt, die bis heute wichtig sind: (1) den Datensatz GazeCapture, (2) die Personalisierung mit Kalibrierdaten und (3) das Zusammenführen von Merkmalen aus mehreren Bildbereichen. Diese Arbeit nutzt dieselben Daten. Statt AlexNet und SVR verwendet sie aber ein ResNet-18 [2] mit einem MLP-Regressionskopf und L1-Verlust. Außerdem bewertet sie die Basismodelle mit einem personendisjunkten Split (`norm_subject`) und nicht mit einer zufälligen Aufteilung einzelner Bilder. Falsche Labels behandelt sie schon bei der Aufnahme: Das Aufnahmeprogramm speichert für jedes Bild die Phase des Ziels (`WAIT` oder `MOVE`).

2.1.3. Einteilung der Verfahren

Man kann Verfahren zur Blickschätzung in drei Klassen einteilen:

1. **Modellbasierte (geometrische) Verfahren** nutzen ein 3D-Modell des Augapfels, Pupillenellipsen und Lichtreflexe auf der Hornhaut. Sie brauchen meist hochaufgelöste Bilder und Infrarotlicht. Bei normalem Tageslicht machen sie oft Fehler.

2. **Merkmalsbasierte Verfahren** suchen geometrische Punkte im Gesicht, zum Beispiel Augenwinkel oder die Mitte der Iris. Daraus berechnen sie den Blick.

3. **Erscheinungsbasierte Verfahren** verwenden das Augen- oder Gesichtsbild selbst als Eingabe. Ein tiefes neuronales Netz sagt direkt 2D-Koordinaten oder 3D-Winkel vorher.

Durch den Fortschritt im Deep Learning sind erscheinungsbasierte Verfahren heute der Standard für uneingeschränkte Bedingungen.

Cheng et al. [3] teilen Verfahren mit entfernter Kamera ebenfalls in drei Gruppen ein: Verfahren mit einem rekonstruierten 3D-Augenmodell, Regression auf 2D-Augenmerkmalen und erscheinungsbasierte Verfahren. Deep-Learning-Ansätze ordnen sie nach vier Gesichtspunkten: Merkmalsextraktion, Netzarchitektur, persönliche Kalibrierung sowie Geräte und

Plattformen. Ein wichtiger Beitrag ist ein Benchmark. Er vereinheitlicht die Vorverarbeitung (Gesichts- und Augenerkennung, Normalisierung der Daten) und die Nachverarbeitung (Umrechnung zwischen 2D-Blickpunkt und 3D-Blickvektor). So kann man Verfahren fair vergleichen. Bei kurzem Abstand zur Kamera oder hoher Auflösung erreichen erscheinungsbasierte Verfahren laut Benchmark etwa 5°. Bei großem Abstand und niedriger Auflösung (zum Beispiel Gaze360 und RT-GENE) sind die Fehler deutlich größer. Als offene Themen nennen die Autoren: (1) robustere Merkmale, die nicht von der Umgebung abhängen, (2) hohe Genauigkeit mit schneller und einfacher Kalibrierung – hier gibt es einen Zielkonflikt zwischen Kalibrierdauer und Genauigkeit – und (3) die Erklärbarkeit der gelernten Merkmale.

Weitere Übersichtsarbeiten geben zusätzlichen Hintergrund. Liu et al. [4] beschreiben den Weg von klassischen Verfahren zu Deep Learning; ich nutze diese Arbeit vor allem als historische Referenz. Pathirana et al. [5] behandeln Deep-Learning-Verfahren für einen und für mehrere Nutzer. Sie vergleichen CNN-Architekturen, von denen einige auf ImageNet vortrainiert sind. Neuere vortrainierte Modelle wie DINOv2, CLIP oder MAE stehen in diesen Übersichten aber nicht im Mittelpunkt. Für Vergleiche mit Benchmarks nutze ich daher vor allem Cheng et al. [3].

2.2. Webcam-basiertes Eye-Tracking: Alltag und Labor

Webcams sind billig und fast überall vorhanden: in Laptops, an Desktop-Rechnern und in Smartphones. Man braucht keine zusätzliche Hardware. Dafür sind die Bedingungen viel schwieriger als bei speziellen Eye-Trackern mit Infrarotlicht:

- **Infrarot-Systeme** nutzen hochauflösende Kameras, Infrarotlicht und Reflexe auf der Hornhaut. Liu et al. [4] nennen für solche Verfahren typische Genauigkeiten von 0,5° bis 2°. Kommerzielle Geräte arbeiten in festen Abständen (zum Beispiel 50 cm bis 80 cm) und erreichen unter optimalen Bedingungen 0,3° bis 0,5°.
- **Webcams im Alltag** liefern Bilder mit niedriger Auflösung und ohne kontrolliertes Licht. Das Umgebungslicht ändert sich, die Lage von Kamera und Bildschirm ist bei jedem Gerät anders, und Abstand und Kopfhaltung sind frei. Liu et al. [4] nennen für erscheinungsbasierte Verfahren typische Fehler über 3°. Der Benchmark von Cheng et al. [3] zeigt etwa 5° bei kurzem Abstand und deutlich mehr bei großem Abstand und niedriger Auflösung.

Zwei bekannte Arbeiten zeigen trotzdem, dass Webcam-Eye-Tracking praktisch nutzbar ist. Papoutsaki et al. [6] stellen *WebGazer* vor, einen Open-Source-Eye-Tracker, der komplett im Browser läuft. WebGazer braucht keine eigene Kalibrierung. Er lernt mit Ridge-Regression laufend aus Mausklicks und Mausbewegungen, welche Augenmerkmale zu welch-

er Bildschirmposition gehören. Die Genauigkeit reicht, um grob den betrachteten Bereich zu erkennen, aber nicht für genaue Analysen. Valliappan et al. [7] trainieren dagegen ein CNN auf GazeCapture vor, passen es mit Kalibrierdaten an und ergänzen ein kleines Regressionsmodell pro Person. Auf Smartphones bei 25 cm bis 40 cm Abstand erreichen sie 0,6° bis 1°. Etwa 30 Sekunden Kalibrierung senken den Fehler von 1,92 cm auf 0,46 cm. Laut den Autoren ist das so genau wie viel teurere mobile Eye-Tracker. Die Bedingungen waren aber günstig (Kopf fast frontal, kurzer Abstand). Beide Arbeiten zeigen: Ohne spezielle Hardware ist die Personalisierung der wichtigste Faktor für die Genauigkeit.

Auch Eigenschaften der Person beeinflussen die Genauigkeit, zum Beispiel Augenform, Gesichtsform, Brille und die gewohnte Kopfhaltung. Allgemeine Modelle haben deshalb bei neuen Personen oft einen systematischen Versatz. Aus diesem Grund ist die Personalisierung ein zentrales Forschungsthema (Abschnitt 2.6). Diese Arbeit untersucht genau solche Bedingungen, unter anderem verschiedene Betriebssysteme (Ubuntu und Windows), Brillen und Hintergründe.

2.3. Große Datensätze zur Blickschätzung

Moderne erscheinungsbasierte Verfahren brauchen große und vielfältige Datensätze. Tabelle 2.1 fasst die für diese Arbeit wichtigen Datensätze zusammen.

- **MPIIGaze** [8] ist einer der ersten standardisierten Datensätze aus dem Alltag. Er enthält 213 659 Gesichtsbilder von 15 Personen. Sie wurden über mehrere Monate bei der normalen Arbeit am Laptop aufgenommen [9]. Die Variante MPIIFaceGaze mit Gesichtsbildern wird meist mit Leave-one-subject-out ausgewertet (jede Person ist einmal die Testperson).
- **GazeCapture** [1] enthält 2 445 504 Bilder von 1474 Personen, aufgenommen per Crowdsourcing auf iPhones und iPads. Licht und Kopfhaltung sind sehr verschieden. GazeCapture ist die Datengrundlage der Basismodelle dieser Arbeit.
- **ETH-XGaze** [10] ist ein hochauflösender Datensatz mit mehreren Kameras. Er enthält mehr als eine Million Bilder von 110 Personen mit sehr großen Kopf- und Blickwinkeln.
- **EVE** [11] enthält gleichzeitig aufgenommene Bildschirminhalte, Kamerabilder und Blickdaten. 54 Personen wurden mit vier Kameras aufgenommen, insgesamt mehr als 12 Millionen Bilder.
- **Gaze360** [12] enthält Aufnahmen von 238 Personen drinnen und draußen. Die 3D-Blicklabels decken sehr viele Kopfhaltungen und Abstände ab (etwa 172 000 Bilder in Training, Validierung und Test).

- **EyeDiap** und **RT-GENE** werden vor allem für Tests über Datensätze hinweg verwendet. RT-GENE enthält unter anderem größere Kameraabstände und am Kopf getragene Geräte.

Tabelle 2.1.: Überblick über ausgewählte Datensätze zur Blickschätzung.

Datensatz	Umfang	Personen	Aufnahmebedingungen
MPIIGaze	213 659 Bilder	15	Laptop-Webcam, Alltag über Monate
GazeCapture	2 445 504 Bilder	1474	Smartphones und Tablets, Crowdsourcing
ETH-XGaze	¿1 Million Bilder	110	Labor mit mehreren Kameras, extreme Kopfhaltungen
EVE	¿12 Millionen Bilder	54	Bildschirm und Kameras, Videosequenzen
Gaze360	ca. 172 000 Bilder	238	Drinnen und draußen, viele Kopfhaltungen und Abstände

2.4. Netzarchitekturen für die Blickregression

2.4.1. Convolutional Neural Networks

CNNs wie ResNet-18/50 [2], VGG oder MobileNet sind die üblichen Merkmalsextraktoren für Augen- und Gesichtsbilder. Ihre lokalen Faltungen erkennen feine Texturen und Kanten in der Augenregion. Auch spezielle Blickmodelle bauen oft auf CNNs auf: FAZE [13] nutzt zum Beispiel einen Encoder-Decoder auf DenseNet-Basis, um Blick, Kopfhaltung und Aussehen voneinander zu trennen. In dieser Arbeit ist ResNet-18 das verwendete Modell.

2.4.2. Vision Transformer

Vision Transformer (ViTs) [14] teilen ein Bild in kleine Felder (Patches). Mit Self-Attention erfassen sie Zusammenhänge über das ganze Bild, zum Beispiel zwischen Gesicht, Kopfdrehung und Augen. UniGaze [15] zeigt, dass mit MAE vortrainierte ViTs (Größen Base, Large und Huge) sehr gute Backbones für die Blickschätzung sind. TTP-GE [16] kombiniert dagegen zwei ResNet-18-Zweige mit einer Transformer-Fusion. GazeCLIP [17] zeigt, dass Transformer nicht immer besser sind: Dort erreicht ein CLIP-vortrainiertes ResNet-50 3,5° auf MPIIFaceGaze, die Variante mit ViT-B/32 nur 3,7°. Ye et al. [18] zeigen dagegen mit *TransGaze*, dass auch einfache ViTs mit großem Vortraining sehr gute Ergebnisse auf GazeCapture und MPIIFaceGaze erreichen. Am besten war dort das selbstüberwachte DINO-Vortraining. Man sollte also CNNs und Transformer gleichermaßen prüfen und nicht eine Architektur für allgemein besser halten.

2.4.3. Vortrainierte Foundation-Modelle

Beim Transfer Learning nutzt man ein Modell, das auf sehr vielen Bildern vortrainiert wurde. Seine Merkmale sind allgemein nutzbar. Man kann den Backbone einfrieren und nur einen kleinen Regressionskopf darauf trainieren. Für diese Arbeit sind vier Arten von Vortraining wichtig:

- **Überwachtes Vortraining auf ImageNet**: der klassische Ausgangspunkt für CNNs wie ResNet [2]. Diese Arbeit verwendet ein solches Modell.
- **DINOv2** [19]: lernt robuste Merkmale selbstüberwacht, also ohne manuelle Labels.
- **CLIP** [20]: lernt Bild und Text gemeinsam durch kontrastives Lernen auf Bild-Text-Paaren.
- **Masked Autoencoders (MAE)** [21]: lernen, indem sie verdeckte Bildteile rekonstruieren.

Wie man diese Merkmale für die Blickschätzung nutzt, beschreibt der nächste Abschnitt.

2.5. Vortrainierte Modelle für die Blickschätzung

2.5.1. Eingefrorener Backbone oder vollständiges Fine-Tuning

Es gibt zwei Grundstrategien, ein vortrainiertes Modell an die Blickschätzung anzupassen. Beim *vollständigen Fine-Tuning* werden alle Gewichte trainiert. Das Modell passt sich so am stärksten an, braucht aber viele Daten. Mit wenigen Daten droht Überanpassung oder *katastrophales Vergessen*: Das Modell verliert das, was es im Vortraining gelernt hat. Bei einem *eingefrorenen Backbone* (zum Beispiel ResNet-18 oder ViT-B/14) bleiben die vortrainierten Gewichte fest. Man trainiert nur einen kleinen Regressionskopf. So gibt es viel weniger trainierbare Parameter, und die allgemeinen Merkmale bleiben erhalten. Das ist wichtig, wenn man nur wenige Kalibrierdaten hat.

Kumar et al. [22] begründen diesen Zielkonflikt theoretisch und mit Experimenten. Sie vergleichen vollständiges Fine-Tuning mit *Linear Probing*, bei dem nur die letzte lineare Schicht trainiert wird. Auf zehn Datensätzen mit Verteilungsverschiebung ist Fine-Tuning auf Daten wie im Training im Mittel um 2 % besser, auf abweichenden Daten aber um 7 % schlechter. Der Grund: Wenn Kopf und Backbone gleichzeitig trainiert werden und der Kopf am Anfang noch zufällig ist, werden die vortrainierten Merkmale verzerrt. Die Autoren schlagen deshalb LP-FT vor: zuerst Linear Probing, dann Fine-Tuning. Für die Personalisierung ist das direkt wichtig. Die Kalibrierdaten einer Person sind wenige und decken die spätere Nutzung nur zum Teil ab. Ein eingefrorener Backbone kann hier die Generalisierung schützen.

Der Regressionskopf ist meist ein mehrschichtiges Perzeptron (MLP). Es bildet die Merkmale, zum Beispiel die 512 Zahlen eines ResNet-18, auf die Koordinaten (x, y) ab. Wichtige Entscheidungen sind die Aktivierung am Ausgang (zum Beispiel Sigmoid, Gauß oder ReLU) und die Verlustfunktion (zum Beispiel L1, Smooth L1 oder Varianten mit Gewichtung nach der Position). Zwischen den beiden Extremen liegen parametereffiziente Verfahren wie Adapter oder LoRA [23] (Abschnitt 2.6.3).

2.5.2. Großes MAE-Vortraining: UniGaze

Qin et al. [15] stellen mit UniGaze ein großes Vortraining speziell für die Blickschätzung vor. Mit Masked Autoencoders lernt das Modell aus etwa 1,6 Millionen normalisierten Gesichtsbildern von mehr als 260 000 Personen. Die Daten kommen aus echten Quellen (VGGFace2, VFHQ, CelebV-Text) und aus synthetischen Datensätzen (FaceSynthetics, SFHQ-T2I, FFHQ-NV, XGaze-Dense). Ein zentrales Ergebnis ist, wie wichtig die Normalisierung der Daten ist. Dabei werden Gesichtsbild und Blicklabel durch Drehen und Skalieren in einen gemeinsamen Raum gebracht. So verschwindet ein großer Teil der Unterschiede durch Kopfhaltung und Abstand. Zhang et al. [24] haben dieses Verfahren als Erste genau untersucht. Eine Variante ohne Skalierungsfaktor verbessert die Genauigkeit je nach Test um 9,5 bis 32,7 %. UniGaze bestätigt das: In den Ablationen (UniGaze-B, nur echte Vortrainingsdaten, Leave-one-dataset-out) senkt die Normalisierung den Winkelfehler auf XTest von 14,56° auf 11,95°. Mit zusätzlichen synthetischen Daten werden 11,78° erreicht.

UniGaze nutzt ViTs der Größen Base, Large und Huge. Beim Vortraining werden 75 % des Bildes verdeckt. ViT-Huge wird 300 Epochen lang vortrainiert. Das dauert etwa 120 h auf vier NVIDIA-H100-GPUs. Danach werden die Modelle mit L1-Verlust auf MPIIFaceGaze, ETH-XGaze, EYEDIAP, Gaze360 und GazeCapture vollständig feinabgestimmt. Die Autoren testen drei Protokolle:

- **Domänengeneralisierung** (Training auf ETH-XGaze oder Gaze360, Test auf MPIIFaceGaze oder dem Bildschirm-Teil von EYEDIAP): UniGaze-H erreicht 5,57° (X→M), 4,65° (X→ECS), 5,65° (G360→M) und 5,86° (G360→ECS). Das ist besser als andere Verfahren, zum Beispiel CLIP-Gaze mit 6,41°, 7,51°, 6,89° und 7,06°.
- **Leave-one-dataset-out** (Training auf vier Datensätzen, Test auf dem fünften): UniGaze-H erreicht 11,29° (XTest), 4,51° (MPIIFaceGaze), 5,47° (GazeCapture), 5,88° (EYEDIAP) und 12,37° (Gaze360).
- **Gemeinsames Training auf allen Datensätzen**: UniGaze-H erreicht 4,46° (XTest), 5,08° (MPIIFaceGaze), 3,20° (GazeCapture), 5,16° (EYEDIAP) und 9,07° (Gaze360).

Die Ablationen zeigen: Mehr Vortrainingsdaten verbessern die Generalisierung, und echte plus synthetische Daten sind besser als nur echte Daten. Auch Normalisierung und viele verschiedene Kopfhaltungen sind wichtig. Wenn man die Vielfalt der Kopfhaltungen einschränkt, steigt der Fehler auf XTest von 11,95° auf 16,70°. Vergleiche mit DINO und FaRL deuten darauf hin, dass die Zusammensetzung der Daten wichtiger sein kann als ihre Menge.

Grenzen. UniGaze lässt einige Fragen offen, die für die Personalisierung mit Webcams wichtig sind. Die Ergebnisse beruhen auf vollständigen Datensätzen und vollständigem Fine-Tuning, nicht auf einer kurzen Kalibrierung mit wenigen Bildern einer Person. Sparsame Strategien wie reines Kopftraining, LoRA oder Adapter stehen nicht im Mittelpunkt. Auch der Einfluss falscher Labels in kurzen Kalibrierungen wird nicht untersucht. Das Vortraining braucht außerdem sehr viel Rechenleistung (ViT-Huge: etwa 120 h auf 4×H100). Offen bleibt auch, ob die Verbesserungen unter echten Webcam-Bedingungen gelten – mit Brille, wechselndem Hintergrund, niedriger Auflösung und Unterschieden zwischen Betriebssystemen.

Bedeutung für diese Arbeit. UniGaze ist eine wichtige Referenz für Vortraining speziell für den Blick. UniGaze untersucht die Generalisierung über viele Personen. Diese Arbeit stellt die ergänzende Frage: Wie gut kann man ein vortrainiertes Modell mit einer kurzen Kalibrierung (hier 30 s bis 120 s, also etwa 180 bis 720 Labels) an eine einzelne Person anpassen? Diese Arbeit verwendet dafür ein auf ImageNet vortrainiertes ResNet-18. UniGaze-, DINOv2- und CLIP-Modelle sind ein naheliegender nächster Schritt (Tabelle 2.2).

2.5.3. Bild-Text-Modelle: CLIP-Gaze und GazeCLIP

Zwei neuere Arbeiten nutzen das Bild-Text-Modell CLIP [20] für die Blickschätzung, aber mit verschiedenen Zielen.

CLIP-Gaze. Yin et al. [25] wollen mit CLIP-Gaze die Domänengeneralisierung verbessern. Das Modell soll in einer neuen Umgebung gut bleiben, ohne gelabelte Daten aus dieser Umgebung. Die Idee: Faktoren, die nichts mit dem Blick zu tun haben, werden durch Texte beschrieben, etwa „An image of a face with {condition}.“ Diese Faktoren bilden drei Gruppen: Aussehen (Identität, Gesichtsausdruck, Haare, Alter usw.), Zubehör (Brille, Hut, Make-up usw.) und Bildqualität (Rauschen, Unschärfe, Licht). Drei Verluste steuern das Training: Ein Distillationsverlust richtet die Blickmerkmale am CLIP-Bildencoder aus. Ein Trennungsverlust macht sie den blickfremden Merkmalen unähnlich. Ein Rangverlust sorgt dafür, dass ähnliche Blicke auch ähnliche Merkmale haben. Zusätzlich passt die *Personalized Context Optimization* (PCO) die Texte an die jeweilige Person an. CLIP bleibt

Tabelle 2.2.: Vergleich von UniGaze und dieser Arbeit.

Kriterium	**UniGaze-H**	**UniGaze-B**	**Diese Arbeit**
Vortrainingsdaten	1,6 Millionen Gesichtsbilder, ¿260 000 Personen	Gleiche Daten	ImageNet (ResNet-18), Basiskopf auf GazeCapture
Backbone	ViT-Huge	ViT-Base	ResNet-18, eingefroren
Fine-Tuning	Gesamtes Modell	Gesamtes Modell	Nur Regressionskopf
Kalibrierdaten	Vollständige Trainingsdatensätze	Vollständige Trainings-datensätze	30 s bis 120 s pro Sitzung (ca. 180–720 Labels)
Evaluation	Über Datensätze hinweg, Domänen-generalisierung	Über Datensätze hinweg, Domänen-generalisierung	Personendisjunkte Basismodelle, personalisierte Webcam-Sitzungen
Falsche Labels	Keine Filterung nach Phase	Keine Filterung nach Phase	`WAIT`/`MOVE`-Protokoll
Rechenaufwand	Ca. 120 h auf 4×H100	Nur für ViT-H angegeben	Kein Vortraining; Personalisierung ca. 2 s auf einer CPU
Winkelfehler (Leave-one-dataset-out)	4,51°–12,37°	4,73°–12,41°	Diagonalfehler in %, nicht direkt vergleichbar

eingefroren. Trainiert wird ein eigenes Netz aus ResNet-18, einem MLP-Filter und einem Regressionskopf. Mit ETH-XGaze und Gaze360 als Quelle und MPIIFaceGaze und EyeDiap als Ziel erreicht CLIP-Gaze über vier Aufgaben im Mittel etwa 6,97°.

GazeCLIP. Wang et al. [17] nutzen grobe Blickinformation aus Text für die überwachte Blickregression. Ein Prompt-Modul bestimmt ohne Training (Zero-Shot) mit Texten wie „A photo of a face gazing [class]“ eine grobe Richtung aus {Front, Down, Left, Right}. Der Bildencoder ist ein CLIP-vortrainiertes ResNet-50 und wird feinabgestimmt. Der Textencoder (ein Transformer mit 12 Schichten) bleibt eingefroren. Ein Cross-Attention-Modul verbindet beide: Text ist die Query, die Bildmerkmale sind Key und Value. Ein MLP mit drei Schichten ($1024 \to 256 \to 128 \to 2$) sagt Pitch und Yaw vorher. GazeCLIP erreicht 3,5° auf MPIIFaceGaze, 4,7° auf EyeDiap und 7,3° auf RT-GENE. Cross-Attention (3,6°) ist besser als einfaches Aneinanderhängen (4,2°) oder Addieren (4,4°). Ein feinabgestimmter Bildencoder mit eingefrorenem Textencoder (3,6°) ist viel besser als zwei eingefrorene Encoder (8,9°). Fehler entstehen vor allem bei schlechtem Licht und wenn Haare oder Brillen die Augen verdecken. Außerdem kann die Zero-Shot-Klassifikation in einer neuen Umgebung die falsche Richtung liefern.

Grenzen und Bedeutung für diese Arbeit. Beide Arbeiten zeigen, dass CLIP-Merkmale für die Blickschätzung nützlich sind. Eine Personalisierung mit kurzer Kalibrierung untersuchen sie aber nicht. Bei CLIP-Gaze bleibt offen, ob eine Anpassung des CLIP-Bildencoders die Schätzung für eine Person verbessert. GazeCLIP braucht vollständiges überwachtes Training auf Benchmarkdaten. Es behandelt weder Personalisierung mit wenigen Bildern noch falsche Kalibrierlabels, und die Daten gibt es laut Veröffentlichung nur auf Anfrage. CLIP-Merkmale wurden in dieser Arbeit noch nicht untersucht; sie sind ein Kandidat für den Ausblick (Tabelle 2.3). Die Winkelfehler dieser Arbeiten sind nur Orientierungswerte. Sie lassen sich nicht direkt mit dem Diagonalfehler dieser Arbeit vergleichen.

2.6. Dateneffiziente Personalisierung

Allgemeine Modelle haben bei neuen Personen oft einen systematischen Versatz. Er entsteht durch Unterschiede im Auge, in der Gesichtsform und in der Kopfhaltung. Park et al. [13] sprechen von anatomischen Unterschieden zwischen Personen, die die Genauigkeit allgemeiner Netze begrenzen. Die Personalisierung soll diese Lücke mit möglichst wenig Kalibrieraufwand schließen. Die folgenden Abschnitte beschreiben klassische Kalibrierung, Few-Shot- und Meta-Learning sowie parametereffiziente Verfahren.

Tabelle 2.3.: Vergleich von CLIP-Gaze, GazeCLIP und dieser Arbeit.

Aspekt	**CLIP-Gaze [25]**	**GazeCLIP [17]**	**Diese Arbeit**
Ziel	Domänengeneralisierung	Überwachte Blickregression	Personalisierung mit Webcam und wenigen Daten
Nutzung von CLIP	Bild- und Textencoder eingefroren	Bildencoder feinabgestimmt, Textencoder eingefroren	Nicht verwendet (Ausblick)
Architektur	ResNet-18, MLP-Filter, Regressionskopf	CLIP-ResNet-50, Cross-Attention, MLP	ResNet-18 (ImageNet), MLP-Kopf
Labels der Zielperson	Keine	Vollständige Trainingsdaten	30 s bis 120 s Kalibrierung
Berichteter Fehler	Ca. 6,97° (Mittel über 4 Aufgaben)	3,5° / 4,7° / 7,3°	Diagonalfehler in %
Datenqualität	Modelliert blickfremde Faktoren	Grobe Richtung aus Text (unsicher)	`WAIT`-Bilder werden entfernt

2.6.1. Kalibrierverfahren

Bei der aktiven Kalibrierung schaut die Person nacheinander auf bekannte Punkte auf dem Bildschirm. Die Koordinaten dieser Punkte sind die Labels für die Anpassung. Typisch sind wenige feste Punkte oder ein Punkt, der sich über den Bildschirm bewegt. iTracker [1] verwendet bis zu 13 feste Punkte und trainiert darauf eine SVR pro Nutzer (Abschnitt 2.1.2). He et al. [26] zeigen, dass gute Verfahren mit nur 2 bis 5 Punkten pro Person besser sind als frühere Verfahren mit mehr als 13 Punkten. Auf Smartphones ist ihr bestes Modell mit höchstens fünf Punkten um 24 bis 26 % genauer als der damalige Stand der Technik und läuft in Echtzeit auf dem Gerät. Sie schlagen außerdem ein Verfahren vor, das nur ungelabelte Gesichtsbilder braucht. Valliappan et al. [7] verwenden neben Punkten eine *Zickzack-Kalibrierung*: Ein Ziel bewegt sich 60 Sekunden lang langsam von links oben nach rechts unten, und die Frontkamera nimmt mit 30 Hz auf. Schon etwa 100 Bilder senken den Fehler fast auf ein Viertel.

Die Verfahren unterscheiden sich nicht nur darin, *wie viele* Daten sie brauchen, sondern auch darin, *welchen Teil* des Modells sie anpassen:

- **Offset-Kalibrierung:** Chen und Shi [27] teilen den Blickwinkel in einen Teil, der für alle Personen gleich ist, und einen persönlichen Versatz (Bias). Sie beobachten, dass der Versatz zwischen Personen größer ist als die Schwankung bei einer Person. Die Kalibrierung braucht dann nur einen einzigen Blickpunkt. Auf MPIIGaze und EYEDIAP ist das Verfahren um bis zu 10 % besser als vergleichbare Modelle. Schon etwa neun Kalibrierbilder senken den Fehler deutlich.

- **Wenige Personenparameter:** Lindén et al. [28] beschreiben persönliche Unterschiede mit dem *Spatial Adaptive Gaze Estimator* (SPAZE) durch wenige latente Parameter. Die Kalibrierung ist dann nur ein kleines Optimierungsproblem und kaum anfällig für Überanpassung. Mit neun Beispielen erreicht SPAZE 2,70° auf MPIIGaze, 14 % besser als der damalige Stand der Technik. Drei Parameter pro Auge reichen, um die persönlichen Unterschiede gut zu beschreiben.
- **Differenzielle Schätzung:** Liu et al. [29] trainieren ein CNN, das den *Unterschied* im Blick zwischen zwei Augenbildern derselben Person vorhersagt. Mit gelabelten Referenzbildern aus der Kalibrierung ergibt sich daraus der Blick im neuen Bild. Weil beide Bilder von derselben Person stammen, heben sich Störungen wie Augenform, Lidschluss oder Licht teilweise auf. Auf EYEDIAP, MPIIGaze und UT-Multiview ist das Verfahren schon mit einem einzigen Kalibrierbild besser als der Stand der Technik.
- **Personalisierung ohne Kalibrierung:** Zhao et al. [30] schlagen das *Episode-based Personalization Network* (EbPN) vor. Ein eigener Zweig des Netzes soll persönliche Abweichungen ohne Kalibrierdaten ausgleichen.

Zusammen zeigen diese Arbeiten: Ein großer Teil des persönlichen Fehlers lässt sich mit wenigen angepassten Parametern erklären. Das unterstützt die Strategie dieser Arbeit, nur einen kleinen Regressionskopf auf einem eingefrorenen Backbone anzupassen.

Cheng et al. [3] nennen einen grundsätzlichen Zielkonflikt: Längere Kalibrierungen machen die Schätzung genauer, sind aber unbequemer. Hohe Genauigkeit mit kurzer, einfacher Kalibrierung ist daher eine offene Forschungsfrage. Ein weiterer Punkt wird oft übersehen: die Qualität der Kalibrierlabels selbst. Wenn ein Ziel springt, folgt der Blick erst nach einer kurzen Verzögerung. Bilder aus dieser Übergangszeit können ein falsches Label haben (Abschnitt 2.9).

2.6.2. Few-Shot-Learning und Meta-Learning: FAZE

Park et al. [13] stellen mit FAZE (*Few-Shot Adaptive Gaze Estimation*) ein einflussreiches Verfahren für die Personalisierung mit wenigen Kalibrierbildern vor. Es verbindet getrennte (entflochtene) Merkmale für Blick, Kopf und Aussehen mit Model-Agnostic Meta-Learning (MAML). FAZE trainiert nicht für jede Person ein neues Modell. Es lernt stattdessen einen Startpunkt, von dem aus wenige Beispiele für eine schnelle Anpassung reichen.

FAZE hat zwei Teile. Ein *Disentangling Transforming Encoder-Decoder* (DT-ED) auf DenseNet-Basis stellt Aussehen, Blickrichtung und Kopfhaltung in getrennten Codes dar. Weil diese Codes mit Drehungen mitgehen (rotationsäquivariant), kann man Änderungen von Blick und Kopf als Drehungen im Merkmalsraum beschreiben. Trainiert wird auf Bildpaaren derselben Person mit einem Rekonstruktionsverlust, einem Blickverlust und

einem Konsistenzterm. Der zweite Teil ist ein Blickschätzer, der mit MAML meta-trainiert wird: Über viele Trainingspersonen lernt er einen Startpunkt, aus dem wenige Gradientenschritte ein persönliches Modell machen. Das soll Überanpassung vermeiden, die beim normalen Fine-Tuning mit sehr wenigen Daten droht.

FAZE wird auf GazeCapture trainiert und auf GazeCapture und MPIIGaze getestet. Mit $k = 9$ Kalibrierbildern liegt der mittlere Fehler auf MPIIGaze bei 3,14°. Für alle $1 \leq k \leq 256$ ist FAZE besser als die verglichenen Verfahren. Auf GazeCapture sinkt der Fehler gegenüber dem Stand der Technik um 19 %, von 3,91° auf 3,18°.

Grenzen und Abgrenzung. FAZE zeigt, dass eine Kalibrierung nicht nur von der Datenmenge abhängt, sondern auch von den vorher gelernten Merkmalen und der Art der Anpassung. Die Merkmale von FAZE kommen aber aus überwachtem Training mit Blickdaten und einem DenseNet. Allgemeine vortrainierte Modelle (DINOv2, MAE, CLIP) werden nicht untersucht. Die Anpassung nutzt Meta-Learning; einfachere Wege wie reines Kopftraining, LoRA oder Adapter stehen nicht im Mittelpunkt. Auch Webcam-typische Probleme wie schwankende Bildqualität, Brillen, Hintergründe und Betriebssysteme werden nicht betrachtet, ebenso wenig das Entfernen von Kalibrierbildern nach der Aufnahmephase. Diese Arbeit greift einige dieser Punkte auf. Da FAZE Winkelfehler angibt und diese Arbeit einen Fehler relativ zur Bildschirmdiagonale, sind die Zahlen nicht direkt vergleichbar.

Tabelle 2.4.: Vergleich von FAZE und dieser Arbeit.

Aspekt	**FAZE [13]**	**Diese Arbeit**
Ziel	Personalisierung der Blickrichtung mit wenigen Bildern	Dateneffiziente Personalisierung der Bildschirmposition mit Webcam
Merkmale	Entflochtene Codes, DenseNet-Encoder-Decoder	ResNet-18, auf ImageNet vortrainiert und eingefroren
Vortraining	Überwacht auf GazeCapture	ImageNet; Basiskopf auf GazeCapture
Anpassung	MAML und Gradientenschritte pro Person	Nur Regressionskopf
Kalibrierdaten	$k = 1$ bis 256 Bilder	30 s bis 120 s (ca. 180–720 Labels)
Labelqualität	Keine Filterung nach Phase	`WAIT`-Bilder über Phasenprotokoll entfernt
Evaluation	GazeCapture und MPIIGaze	18 Webcam-Sitzungen unter wechselnden Bedingungen
Berichteter Fehler	3,14° (MPIIGaze, $k = 9$)	Diagonalfehler in %, nicht direkt vergleichbar

Yu et al. [31] erweitern die Anpassung mit wenigen Bildern: Aus wenigen Referenzbildern einer Person erzeugen sie mit Blickumlenkung (Gaze Redirection) zusätzliche Trainingsbilder und passen damit ein allgemeines Modell an.

2.6.3. Parametereffizientes Fine-Tuning (PEFT)

Parametereffiziente Verfahren passen nur einen kleinen Teil der Parameter an. Der vortrainierte Backbone bleibt fast unverändert. Das spart Speicher und Rechenzeit und senkt die Gefahr der Überanpassung, wenn es nur wenige Kalibrierdaten gibt. Für diese Arbeit sind drei Varianten wichtig:

- **Reines Kopftraining (nur FC):** Der Backbone bleibt komplett eingefroren, nur der Regressionskopf wird trainiert. Man kann die Merkmale des Backbones einmal berechnen und speichern. Deshalb ist dieses Verfahren besonders schnell. Diese Arbeit verwendet es.
- **Adapter** [32]: Kleine zusätzliche Module werden zwischen die Schichten eines eingefrorenen Netzes gesetzt. Nur sie werden trainiert.
- **Low-Rank Adaptation (LoRA)** [23]: Die vortrainierten Gewichtsmatrizen W bleiben fest. Trainiert wird nur eine Änderung $\Delta W = BA$ aus zwei schmalen Matrizen mit niedrigem Rang r. So gibt es viel weniger trainierbare Parameter.

In der Blickschätzung wurden solche Verfahren bisher kaum systematisch untersucht. Eine Ausnahme ist TPGaze [33]. Es überträgt das Prompt-Tuning aus der Sprachverarbeitung: Ein zusätzlicher Prompt mit weniger als 1 % der Parameter eines ResNet-18 wird zur Testzeit ohne Labels angepasst. Meta-Learning sorgt dafür, dass diese Anpassung den Blickfehler wirklich senkt (Abschnitt 2.8). Bao und Lu [34] zeigen außerdem, dass gerade die letzte vollständig verbundene Schicht, die viele Bildmerkmale auf den Blick abbildet, leicht überangepasst wird. Auch beim reinen Kopftraining ist der Aufbau des Kopfes also wichtig. Die anderen hier betrachteten Arbeiten nutzen vollständiges Fine-Tuning (UniGaze, GazeCLIP), Meta-Learning (FAZE) oder ein eingefrorenes Modell, das ein separat trainiertes Netz steuert (CLIP-Gaze). Diese Arbeit untersucht reines Kopftraining für die Personalisierung mit kurzen Webcam-Kalibrierungen. Adapter und LoRA bleiben für spätere Arbeiten.

2.7. Domänenwechsel, Generalisierung und Evaluationsprotokolle

2.7.1. Evaluation mit unbekannten Personen

Wie gut ein Modell auf neue Personen generalisiert, hängt stark vom Evaluationsprotokoll ab. Bei einer zufälligen Aufteilung einzelner Bilder (`norm_random`) können Bilder derselben Person in Training und Test landen. Das Modell kann dann Merkmale dieser Person ausnutzen, und die Generalisierung wird überschätzt. Bei einem personendisjunkten Split (`norm_subject`), etwa Leave-one-subject-out, gibt es dieses Datenleck nicht. Man misst

dann die Leistung auf wirklich unbekannten Personen. In den Basisexperimenten dieser Arbeit auf GazeCapture ergibt `norm_subject` einen Testfehler von 28,39 % und `norm_random` 27,72 %. Der größte Teil dieses Unterschieds kommt aber von den unterschiedlichen Testdaten: Auch die Konstant-Baseline ist auf `norm_random` niedriger (27,66 % statt 28,46 %, siehe Kapitel 4).

Davon zu unterscheiden sind Tests über Datensätze hinweg, zum Beispiel Leave-one-dataset-out in UniGaze [15]. Dort wird ein ganzer Datensatz zurückgehalten. Beide Protokolle messen also verschiedene Arten der Generalisierung: auf unbekannte Personen in derselben Umgebung bzw. auf unbekannte Aufnahmebedingungen.

2.7.2. Domänenadaption und Domänengeneralisierung

Wenn sich Trainings- und Testdaten in Personen, Kameras oder Umgebung unterscheiden, werden erscheinungsbasierte Verfahren deutlich schlechter. Es gibt zwei Gegenstrategien. Die Domänenadaption nutzt (meist ungelabelte) Daten aus der Zielumgebung. Die Domänengeneralisierung braucht gar keine Zieldaten.

CRGA. Wang et al. [35] schlagen mit *Contrastive Regression Gaze Adaptation* (CRGA) eine unüberwachte Domänenadaption vor. Ein Modul für *Contrastive Domain Generalization* (CDG) lernt stabile Merkmale in der Quelldomäne: Merkmale mit ähnlichem Blick werden zusammengezogen, Merkmale mit sehr unterschiedlichem Blick auseinandergeschoben. Ein Modul für *Contrastive Self-training Adaptation* (CSA) lernt danach mit Pseudo-Labels in der Zieldomäne. Mit ETH-XGaze und Gaze360 als Quelle und MPIIGaze, RT-GENE, GazeCapture und EyeDiap als Ziel verbessern die Autoren die Baseline um 40,4 %, 34,7 %, 55,8 % und 34,3 % (Quelle ETH-XGaze) bzw. um 31,7 %, 30,5 %, 32,9 % und 23,8 % (Quelle Gaze360).

CRGA löst also ein anderes Problem: die Anpassung an eine neue Umgebung ohne gelabelte Zieldaten. Diese Arbeit nutzt dagegen wenige gelabelte Kalibrierdaten der Zielperson. Damit ist überwachtes Training mit echten Labels möglich, nicht nur mit Pseudo-Labels. Selbsttraining mit Pseudo-Labels kann außerdem einen Bestätigungsfehler (Confirmation Bias) erzeugen: Falsche Vorhersagen werden im weiteren Training verstärkt. Zudem geht es hier um Unterschiede zwischen Personen und nicht um den Wechsel zwischen Benchmark-Datensätzen. CRGA ist daher verwandte Arbeit zur Robustheit, aber keine direkt vergleichbare Baseline.

Domänengeneralisierung. Verfahren wie PureGaze [36] oder CLIP-Gaze [25] (Abschnitt 2.5.3) versuchen, blickfremde Faktoren schon im Training aus den Merkmalen zu entfernen. Dann funktioniert das Modell auch ohne Zieldaten in neuen Umgebungen.

UniGaze [15] zeigt, dass großes blickspezifisches Vortraining diese Verfahren bei Tests über Datensätze hinweg übertreffen kann (Abschnitt 2.5.2).

2.7.3. Blickschätzung für mehrere Personen

Zhang et al. [37] stellen mit GazeOnce ein einstufiges Verfahren vor. Es schätzt den Blick aller Personen in einem Bild in einem einzigen Durchlauf und bleibt auch bei mehr als zehn Gesichtern echtzeitfähig. Neben der Blickrichtung sagt es auch Gesichtsrahmen (Bounding Boxes) und Gesichtspunkte (Landmarken) vorher. Ein selbstüberwachter Verlust prüft, ob die geschätzte 3D-Richtung zu ihren 2D-Projektionen passt. Für das Training erzeugen die Autoren den synthetischen Datensatz MPSGaze: Sie setzen Gesichter aus ETH-XGaze mit bekanntem Blick in passende Gesichter aus WiderFace ein. Laut den Autoren ist GazeOnce genauer und schneller als bestehende Verfahren, besonders bei vielen Gesichtern.

GazeOnce löst damit ein anderes Problem: den Blick vieler Personen in freien Szenen, nicht die Personalisierung für einen Webcam-Nutzer. Methodisch interessant ist die Konsistenz zwischen 3D-Richtung und 2D-Projektion. MPSGaze zeigt außerdem, dass synthetische Trainingsdaten sich von echten Webcam-Bildern unterscheiden können.

2.7.4. Unterschiede bei Geräten und Umgebung

Neben Unterschieden zwischen Datensätzen beeinflussen viele Faktoren von Gerät und Umgebung die Genauigkeit: Brille und Verdeckungen, Hintergrund und Licht, Bildauflösung und Kameraabstand sowie die Lage von Kamera und Bildschirm. Die Literatur gibt einige Hinweise:

- Cheng et al. [3] beobachten deutlich größere Fehler bei großem Abstand und niedriger Auflösung (Gaze360, RT-GENE).
- GazeCLIP [17] nennt schlechtes Licht und Verdeckung durch Haare oder Brille als typische Fehlerfälle.
- UniGaze [15] betont, wie wichtig viele verschiedene Kopfhaltungen und ein großer Bereich der Blicklabels sind.
- CLIP-Gaze [25] modelliert Zubehör und Bildqualität ausdrücklich als blickfremde Faktoren.

Systematische Untersuchungen dieser Faktoren bei der Personalisierung mit kurzen Webcam-Kalibrierungen fehlen aber weitgehend. Diese Arbeit betrachtet deshalb über mehrere Sitzungen auch Betriebssystem (Ubuntu und Windows), Brille, Hintergrund und Kalibrierdauer (siehe Kapitel 4).

2.8. Anpassung zur Testzeit

Bei der Anpassung zur Testzeit (Test-Time Adaptation, TTA) wird ein fertig trainiertes Modell erst während der Nutzung an die Person oder die Umgebung angepasst, meist mit wenigen oder gar keinen Labels. Anders als bei der klassischen Kalibrierung (Abschnitt 2.6.1) soll der Nutzer dabei möglichst wenig selbst tun müssen.

2.8.1. TTP-GE: selbstüberwachte und schwach überwachte Anpassung

Nagpure et al. [16] stellen mit TTP-GE (*Test-Time Personalized Gaze Estimation*) ein zweistufiges Verfahren für die Blickschätzung im Auto vor. Es baut auf GazeDPTR V2 auf. Diese Architektur hat zwei Zweige: einen für das Originalbild des Gesichts und einen für ein normalisiertes Gesichtsbild. Ein Transformer führt die Merkmale zusammen. Mit Positionsinformation aus einer Punktprojektion kann das Modell zusätzlich Blickzonen erkennen.

Das Meta-Training nutzt zwei Schleifen auf Basis von MAML. In der inneren Schleife passt ein selbstüberwachter Konsistenzverlust (im Stil von BYOL++) das Modell an neue Personen an – ohne Blicklabels. Die äußere Schleife optimiert Blickregression und Zonenerkennung mit gelabelten Daten. Zur Testzeit gibt es zwei Schritte: Zuerst passt sich das Modell selbstüberwacht an ungelabelte Bilder der Person an; das hilft bei veränderten Bildbedingungen. Danach wird die Zonenerkennung mit wenigen Zonenlabels angepasst (zum Beispiel Spiegel, Armaturenbrett, Windschutzscheibe). Diese Labels werden in einer kurzen Einrichtungsphase gesammelt.

Auf IVGaze (44 705 Bilder von 125 Personen) erreicht die Kombination einen mittleren Fehler von 6,00°. Die Baseline liegt bei 6,78°, nur schwach überwachte Anpassung bei 6,13° und nur selbstüberwachte Anpassung bei 6,72°. Der Anteil der Schätzungen mit weniger als 6° Fehler steigt von 49,95 % auf 58,2 %. Die beiden Schritte ergänzen sich also: Die Selbstüberwachung behandelt die Bildbedingungen, die Zonenlabels liefern persönliche Information.

2.8.2. Weitere labelsparende Verfahren

Einige neuere Arbeiten verfolgen ähnliche Ziele mit anderen Mitteln:

- **TPGaze** [33] passt zur Testzeit nur einen meta-gelernten Prompt an. Er hat weniger als 1 % der Parameter eines ResNet-18. Als Ziel ohne Labels reicht ein einfacher Symmetrieverlust. Laut den Autoren ist die Anpassung bis zu zehnmal schneller als bei anderen Verfahren. Getestet wird in vier Szenarien über Datensätze hinweg.

- **ELF-UA** [38] behandelt die Anpassung *ohne Labels* als eigenes Problem: Wenige ungelabelte Bilder der Person reichen. Das Meta-Learning nutzt eine Schranke aus der Domänenadaption als Verlustfunktion.
- **PIGaze** [39] gewinnt Kalibrierdaten im Auto automatisch: Der Blick auf bekannte Zonen wie Spiegel und Windschutzscheibe muss geometrisch zusammenpassen (Selbstkalibrierung). Ein vortrainiertes Basismodell ist ein *eingefrorener* Merkmalsextraktor. Darauf lernt ein kleiner Adapter die persönliche Abbildung. Auf IVGaze und LBW ist PIGaze besser als Fine-Tuning.

Grenzen und Abgrenzung. Diese Verfahren wollen den Kalibrieraufwand mit ungelabelten oder schwach gelabelten Daten senken. Sie schätzen meist 3D-Blickrichtungen und arbeiten bei TTP-GE und PIGaze im Auto mit festen Zonen. Ihre Anpassung (Meta-Training, Online-Updates, EMA-Zielnetze) ist deutlich aufwendiger als einfaches Kopftraining. Allgemeine vortrainierte Modelle wie DINOv2, CLIP oder MAE werden nicht als Backbone untersucht. Diese Arbeit verfolgt den ergänzenden Weg einer *überwachten* Personalisierung mit Bildschirmkoordinaten aus einer kurzen Kalibrierung. Interessant ist aber, dass PIGaze – wie diese Arbeit – einen eingefrorenen Backbone mit einem kleinen, persönlich trainierten Modul kombiniert.

Tabelle 2.5.: Vergleich von TTP-GE und dieser Arbeit.

Aspekt	**TTP-GE [16]**	**Diese Arbeit**
Ansatz	Anpassung zur Testzeit mit ungelabelten Bildern und wenigen Zonenlabels	Überwachte Personalisierung mit gelabelten Kalibrierdaten
Backbone	GazeDPTR V2: zwei ResNet-18-Zweige mit Transformer-Fusion	ResNet-18 (ImageNet), eingefroren
Labels	Ungelabelte Bilder und neun Blickzonen	Bildschirmkoordinaten; `WAIT`-Bilder werden entfernt
Anpassung	Meta-Training, Online-Updates, EMA-Zielnetz	Nur Regressionskopf
Anwendung	Fahrerüberwachung im Auto	Webcam vor einem Bildschirm
Ergebnis	6,00° auf IVGaze	Diagonalfehler in %, nicht direkt vergleichbar

2.9. Datenqualität, falsche Labels und zeitlicher Ablauf

Überwachte Personalisierung setzt voraus, dass jedes Kalibrierbild das richtige Label hat, also den Punkt, auf den die Person in diesem Moment wirklich schaut. In der Praxis

stimmt das nur ungefähr. Abweichungen zwischen Label und echtem Blick nennt man *Label-Rauschen*. Bei kurzen Kalibrierungen ist es besonders schädlich, weil jedes falsche Bild einen großen Anteil der Trainingsdaten ausmacht.

2.9.1. Datenqualität im Eye-Tracking

In der Eye-Tracking-Forschung beschreibt man die Datenqualität meist mit drei Größen: *Genauigkeit* (systematische Abweichung vom wahren Blickpunkt), *Präzision* (Streuung wiederholter Messungen) und *Datenverlust*. Niehorster et al. [40] geben einen aktuellen Überblick, wie man diese Größen misst, und stellen Werkzeuge dafür bereit. Bei lernenden Verfahren kommt eine vierte Größe dazu: die Qualität der *Trainingslabels*. Falsche Labels verfälschen nicht nur die Auswertung, sondern auch das gelernte Modell.

2.9.2. Reaktionszeit und Sakkaden

Wenn ein Kalibrierziel an einer neuen Stelle erscheint, folgt der Blick nicht sofort. Bis die Blickbewegung (Sakkade) beginnt, vergeht eine kurze Reaktionszeit. Valliappan et al. [7] messen auf Smartphones im Mittel 210 ms (Median 167 ms). Das passt zu früheren Studien mit 200 ms bis 250 ms. Danach kommen die Sakkade selbst und eventuell kleine Korrekturen, bis das Auge ruhig auf dem Ziel bleibt.

Bilder aus dieser Übergangszeit haben das Label der *neuen* Position. Der Blick ist aber noch auf der *alten* Position oder irgendwo dazwischen. Bei 30 fps betrifft allein eine Reaktionszeit von etwa 200 ms rund sechs Bilder pro Sprung. Je öfter das Ziel springt, desto mehr Bilder haben ein falsches Label.

2.9.3. Filtern schon bei der Aufnahme

Eine einfache Gegenmaßnahme: Man verwirft die Bilder direkt nach einem Zielwechsel. Valliappan et al. [7] verwerfen zum Beispiel immer die ersten 800 ms nach jedem neuen Ziel. Ein festes Zeitfenster ist leicht umzusetzen. Es beruht aber auf einer Annahme über die Dauer des Übergangs, und seine Wirkung wird meist nicht eigens untersucht.

Diese Arbeit geht einen ähnlichen, aber genaueren Weg. Das Aufnahmeprogramm speichert für jedes Bild die aktuelle *Phase* des Ziels: `WAIT` (eine kurze Wartezeit von 0,5 s direkt nach einem Sprung) oder `MOVE` (das Ziel bewegt sich danach 4,5 s lang). So kann man die `WAIT`-Bilder eindeutig und reproduzierbar erkennen und vor dem Training entfernen. In den Aufnahmen betrifft das etwa 10 % der Bilder. Anders als ein festes Zeitfenster erlaubt dieses Protokoll einen direkten Vergleich: Modelle mit und ohne bereinigte Daten unter sonst gleichen Bedingungen.

2.9.4. Filtern nach der Aufnahme und robuste Lernverfahren

Andere Verfahren erkennen falsche Labels erst *nach* der Aufnahme mit statistischen oder modellbasierten Kriterien.

DMRF. Jiang et al. [41] schlagen mit *Dynamic Multi-model Regression Filtering* (DMRF) ein allgemeines Verfahren gegen falsche Labels in Regressionsaufgaben vor. Es löst ein Henne-Ei-Problem: Falsche Labels verzerren die Modelle, mit denen man das Rauschen schätzt, und verzerrte Modelle schätzen das Rauschen falsch. DMRF hat drei Teile:

1. eine *Aufwärmfilterung* mit einer Verteilungsfunktion der geschätzten Fehler,
2. eine *dynamische Modellauswahl* nach einem Korrelationskriterium,
3. einen *Rückhol-Schritt* (Data Recall), der Beispiele mit kleinem Verlust wieder zu den sauberen Daten nimmt.

Neben 15 Benchmarks aus UCI und KEEL und einer Altersschätzung wird DMRF in das Blickverfahren L2CS-Net eingebaut. Auf einem vorverarbeiteten MPIIGaze-Datensatz mit 45 000 Bildern (Leave-one-person-out) sinkt der mittlere Fehler von 3,92° auf 3,85°. Dabei bleiben 74,3 % der Daten erhalten. Der Code ist öffentlich.

SeeTN. Peng et al. [42] untersuchen als Erste systematisch, wie falsche Labels die Domänengeneralisierung in der Blickschätzung beeinflussen. In einem Vorversuch verfälschen sie 10, 20 und 30 % der Gaze360-Labels mit starkem Gaußrauschen. Der Fehler einer ResNet-18-Baseline auf MPIIGaze steigt dadurch von 7,94° auf 12,17° (20 %) bzw. 14,38° (30 %). Ihr Verfahren *See-Through-Noise* baut einen semantischen Merkmalsraum auf. Es erkennt falsche Beispiele daran, dass Merkmal und Label nicht zusammenpassen, und überträgt Blickinformation von sauberen auf falsche Beispiele.

Vorher oder nachher filtern? Verfahren wie DMRF und SeeTN erkennen falsche Labels aus den Daten selbst. Sie finden daher auch unerwartete Fehlerquellen. Sie brauchen aber genug Beispiele für eine sichere Schätzung, rechnen aufwendiger und wurden bisher nur auf großen Datensätzen getestet, nicht auf kurzen Kalibrierungen einer Person. Das Phasenprotokoll erfasst dagegen nur eine *bekannte* Fehlerquelle, die durch den Ablauf der Aufnahme entsteht – dafür aber eindeutig und ohne zusätzliches Modell. Beide Ansätze ergänzen sich (Tabelle 2.6).

Tabelle 2.6.: Vergleich von nachträglicher Filterung (DMRF) und Phasenprotokoll.

Kriterium	**DMRF [41]**	**Diese Arbeit**
Strategie	Nachher: Fehler schätzen und filtern	Vorher: Phase bei der Aufnahme speichern
Erkennung	Modellvorhersagen, Fehlermuster, dynamische Modellauswahl	`WAIT`/`MOVE`-Label pro Bild
Erfasste Fehler	Alle statistisch auffälligen Beispiele	Nur die Übergangszeit nach einem Sprung
Erhaltene Daten	74,3 % (MPIIGaze)	Ca. 90 %
Anwendung	Viele Personen, Leave-one-person-out	Webcam-Kalibrierung einer Person
Zusatzaufwand	Mehrere Regressionsmodelle, iteratives Training	Kein zusätzliches Modell
Ergebnis	$3{,}92° \rightarrow 3{,}85°$	Diagonalfehler in %, nicht direkt vergleichbar

2.10. Vergleich und kritische Diskussion

Tabelle 2.7 fasst die wichtigsten Arbeiten dieses Kapitels zusammen. Die Spalten sind: Ziel, verwendete Merkmale, Art der Personalisierung und Umgang mit der Labelqualität.

Kritische Diskussion. Aus dem Vergleich ergeben sich vier Beobachtungen:

1. **Merkmale und Personalisierung werden getrennt untersucht.** Arbeiten zu großen vortrainierten Modellen (UniGaze, CLIP-Gaze, GazeCLIP) zielen auf Generalisierung über Datensätze hinweg und nutzen vollständiges Fine-Tuning mit vielen Daten. Arbeiten zur Personalisierung (FAZE, SPAZE, Chen & Shi) nutzen dagegen CNNs, die speziell für den Blick trainiert wurden. Wie gut sich allgemeine vortrainierte Modelle mit wenigen Kalibrierdaten personalisieren lassen, ist offen.

2. **Wenige Parameter reichen oft.** Offset-Kalibrierung, wenige Personenparameter, Prompt-Tuning und eingefrorene Modelle mit Adaptern (PIGaze) zeigen übereinstimmend: Ein großer Teil des persönlichen Fehlers lässt sich mit wenigen angepassten Parametern erklären. Kumar et al. [22] liefern dafür eine allgemeine Begründung.

3. **Falsche Labels werden meist erst nachher behandelt.** DMRF und SeeTN filtern oder gewichten falsche Labels in großen Datensätzen. Fehler durch den Ablauf kurzer Kalibrierungen werden, wenn überhaupt, mit festen Zeitfenstern behandelt (Valliappan et al.), ohne ihren Einfluss genau zu untersuchen.

Tabelle 2.7.: Vergleich der wichtigsten Arbeiten. Die Fehler stammen aus verschiedenen Protokollen und Metriken und sind nicht direkt vergleichbar.

Arbeit	**Ziel**	**Merkmale**	**Personalisierung**	**Labelqualität**	**Ergebnis**
iTracker [1]	2D-Blick, mobil	CNN-Zweige ähnlich AlexNet	SVR, bis zu 13 Punkte	–	1,34 cm (Smartphone)
FAZE [13]	Wenige Bilder	DenseNet-DT-ED, überwacht	MAML, $k \leq 256$	–	3,14° ($k = 9$)
SPAZE [28]	Wenige Bilder	CNN, latente Personenparameter	Kleines Optimierungsproblem	–	2,70° ($k = 9$)
Chen & Shi [27]	Offset-Kalibrierung	CNN	Versatz aus einem Blickpunkt	–	bis zu 10 % besser
Valliappan [7]	Smartphone	CNN, vortrainiert auf GazeCapture	Fine-Tuning + Regression, ca. 30 s	Feste 800-ms-Verwerfung	0,46 cm
CRGA [35]	Unüberwachte DA	ResNet (ImageNet)	Pseudo-Labels	–	bis zu 55,8 % besser
UniGaze [15]	Generalisierung	MAE-ViT, 1,6 Millionen Gesichter	Vollständiges Fine-Tuning	–	4,65–5,86° (DG)
CLIP-Gaze [25]	Generalisierung	CLIP (eingefroren) + ResNet-18	–	Blickfremde Faktoren	6,97° (DG)
GazeCLIP [17]	Überwachte Regression	CLIP-ResNet-50 (feinabgestimmt)	–	–	3,5° (MPIIFaceGaze)
TTP-GE [16]	Anpassung zur Testzeit	GazeDPTR V2	Selbst- + schwach überwacht	–	6,00° (IVGaze)
TPGaze [33]	Anpassung zur Testzeit	ResNet-18	Prompt (¡1 % der Parameter)	–	–
DMRF [41]	Falsche Labels filtern	L2CS-Net	–	Nachher, modellbasiert	3,85° (MPIIGaze)
SeeTN [42]	Robuste Generalisierung	ResNet-18/50	–	Nachher, merkmalsbasiert	–
Diese Arbeit	Webcam-Personalisierung	ResNet-18 (ImageNet), eingefroren	Nur Kopf; 30 s bis 120 s	Vorher, Phasenprotokoll	Diagonalfehler in %

4. **Die Ergebnisse sind kaum vergleichbar.** Die Arbeiten geben Winkel in Grad, Abstände in Zentimetern oder relative Verbesserungen an und nutzen verschiedene Datensätze, Splits und Protokolle. Einen Winkelfehler kann man nur mit bekannter Geometrie (Abstand, Bildschirmgröße) in einen Bildschirmfehler umrechnen. Die Metrik dieser Arbeit, der euklidische Fehler relativ zur Bildschirmdiagonale, ist daher nicht direkt mit der Literatur vergleichbar. Die Zahlen in Tabelle 2.7 dienen der Orientierung, nicht als Rangliste.

2.11. Forschungslücken und Bezug zu dieser Arbeit

2.11.1. Forschungslücken

Aus dem Stand der Technik ergeben sich vier Forschungslücken:

F1: Dateneffiziente Personalisierung vortrainierter Modelle. Große vortrainierte Modelle (ImageNet-CNNs, DINOv2, CLIP, MAE oder UniGaze) wurden für die Blickschätzung bisher vor allem mit vollständigem Fine-Tuning auf großen Datensätzen untersucht. Es fehlen Untersuchungen, wie gut sie sich mit nur wenigen Minuten Webcam-Kalibrierung an eine Person anpassen lassen.

F2: Sparsame Anpassung für die Personalisierung. Mehrere Arbeiten zeigen, dass wenige angepasste Parameter den größten Teil des persönlichen Fehlers erfassen. Trotzdem wurden reines Kopftraining, Adapter und LoRA für die Personalisierung in der Blickschätzung kaum systematisch verglichen.

F3: Labelqualität kurzer Kalibrierungen. Reaktionszeit und Zielwechsel erzeugen Kalibrierbilder mit falschem Label. Bisherige Arbeiten behandeln falsche Labels meist nachher in großen Datensätzen oder verwerfen feste Zeitfenster. Wie sich ein genaues, protokollbasiertes Entfernen solcher Bilder auf die Personalisierung auswirkt, wurde noch nicht untersucht.

F4: Realistische Evaluation unter Webcam-Bedingungen. Viele Ergebnisse stammen aus Benchmarks mit kontrollierten Bedingungen oder aus zufälligen Splits auf Bildebene, die die Generalisierung überschätzen können. Wie Betriebssystem, Brille, Hintergrund und Kalibrierdauer die Personalisierung beeinflussen, ist kaum dokumentiert.

2.11.2. Was diese Arbeit beiträgt

Diese Arbeit bearbeitet die Lücken mit einer einfachen und reproduzierbaren Personalisierungspipeline für Webcam-Eye-Tracking:

- **Eingefrorener Backbone mit kleinem Regressionskopf (F1, F2):** Ein auf ImageNet vortrainiertes ResNet-18 bleibt eingefroren. Seine Merkmale werden gespeichert, und nur ein kleiner MLP-Kopf wird pro Person trainiert. Ich vergleiche außerdem, ob das Trainieren weiterer Schichten hilft. Andere Backbones (DINOv2, CLIP, MAE, UniGaze) und Adapter oder LoRA sind der nächste Schritt (Kapitel 6).
- **Phasenprotokoll und Bereinigung (F3):** Das Aufnahmeprogramm speichert die Phasen `WAIT` und `MOVE`. So lässt sich die Wirkung des Entfernens von Übergangsbildern kontrolliert messen: Modelle mit ursprünglichen und mit bereinigten Daten werden unter sonst gleichen Bedingungen verglichen.
- **Systematische Ablationen:** Batchgröße, Lernrate, trainierbare Schichten, Aktivierung, Verlustfunktion (auch ein nach Position gewichteter L1-Verlust) und Datenmenge werden mit wiederholten Läufen und Konfidenzintervallen untersucht.
- **Realistische Evaluation (F4):** Die Basismodelle werden mit dem personendisjunkten Split `norm_subject` bewertet. Die Personalisierung wird auf 18 Sitzungen mit wechselndem Betriebssystem, Brille, Hintergrund und Kalibrierdauer getestet, und zwar mit zufälligem und mit zeitlichem Split.

Kapitel 3 beschreibt die Methodik, Kapitel 4 die Ergebnisse. Kapitel 5 beantwortet damit die Forschungsfragen aus Kapitel 1.

3. Methodik

Dieses Kapitel gibt einen Überblick über das Vorgehen: Daten (Abschnitte 3.1 und 3.2), Vorverarbeitung (Abschnitt 3.3), Modell und Training (Abschnitt 3.4), Auswertung (Abschnitt 3.5) und Implementierung (Abschnitt 3.6). Die genauen Einstellungen der Experimente stehen in Abschnitt 4.1.

3.1. Datensätze

Die Arbeit nutzt zwei Datenquellen:

- **GazeCapture** [1] für die Basismodelle. Die verwendete Version enthält 347 085 gelabelte Bilder von 262 Personen auf iPhones und iPads. Es gibt zwei Splits: `norm_subject` (keine Person in Training und Test zugleich) und `norm_random` (zufällig auf Bildebene). Für die Experimente werden 1000 Trainings- und 200 Testbilder zufällig gezogen, dazu 200 Validierungsbilder einer weiteren Person.
- **Eigene Aufnahmen** für die Personalisierung. Eine Person (der Autor) hat 27 Kalibriersitzungen mit einer Webcam aufgenommen. 18 davon werden ausgewertet (Abschnitt 3.2).

3.2. Aufnahme der Personalisierungsdaten

Ein Aufnahmeprogramm zeigt einen roten Punkt auf dem Bildschirm, und die Person folgt ihm mit den Augen. Gleichzeitig nimmt die Webcam Bilder auf. Zu jedem Bild wird die Position des Punktes gespeichert, und zwar 6 Labels pro Sekunde. Die Labels liegen in der Datei `norm_labels.csv` jeder Sitzung; die Positionen sind schon auf $[0,1]^2$ normiert. Es gibt drei verwendete Generationen des Programms:

- **Gen 1** (Sitzungen 01–04): Der Punkt bewegt sich gleichmäßig und ohne Sprünge über den ganzen Bildschirm.
- **Gen 2** (Sitzungen 05–09): Der Punkt springt alle 1,4 bis 2,7 Sekunden an eine zufällige neue Stelle. Die Aufnahmen sind kurz (etwa 30 s).

- **Gen 4** (Sitzungen 19–27): Die Aufnahme besteht aus Abschnitten von 5 s. Jeder Abschnitt beginnt mit 0,5 s `WAIT` an einem neuen Startpunkt; danach bewegt sich der Punkt 4,5 s lang (`MOVE`). 90 % der Startpunkte liegen in den Ecken. Das Programm speichert die Phase zu jedem Bild.

Gen 3 (Sitzungen 10–18) wurde verworfen, weil das Modell daraus nichts lernen konnte. Die Sitzungen unterscheiden sich außerdem in Licht, Hintergrund, Brille, Dauer (90 oder 120 s in Gen 4) und Betriebssystem (Ubuntu oder Windows). Tabelle 4.12 zeigt alle Bedingungen.

[**Noch zu schreiben:** Kameramodell, Auflösung, Bildschirmgröße und ungefährer Abstand zum Bildschirm ergänzen.]

3.3. Vorverarbeitung

- **Zielwerte:** Die Blickziele von GazeCapture werden durch Breite und Höhe des jeweiligen Bildschirms geteilt. So liegen alle Ziele im Einheitsquadrat $[0,1]^2$, unabhängig vom Gerät (Abschnitt 4.1.5).
- **Bilder:** Das ganze Kamerabild wird auf 224×224 Pixel skaliert und mit den Mittelwerten und Standardabweichungen von ImageNet normiert. Es gibt keinen Gesichts- oder Augenausschnitt.
- **Bereinigte Labels:** Für Gen 4 kann man die `WAIT`-Bilder entfernen. Das Skript `session_design.py --write-clean` schreibt dafür die Datei `norm_labels_clean.csv`.

3.4. Modell und Training

Das Modell besteht aus zwei Teilen (Abschnitt 4.1.1):

1. einem **ResNet-18**, auf ImageNet vortrainiert, als Merkmalsextraktor. Es liefert 512 Merkmale pro Bild.
2. einem **Regressionskopf** (MLP $512 \to 512 \to 256 \to 128 \to 2$, ReLU, Dropout, Sigmoid am Ausgang), der daraus die Position (x,y) berechnet.

Trainiert wird mit L1-Verlust, AdamW (Lernrate 10^{-4}, Weight Decay 10^{-5}), Batchgröße 32 und Early Stopping. Das Vorgehen hat zwei Stufen:

1. **Basismodell (E0):** Der Kopf wird auf GazeCapture trainiert (`train_base.py`). Von fünf Seeds wird der Lauf mit dem kleinsten Validierungsfehler behalten.

2. **Personalisierung (E1–E3):** Der Kopf des Basismodells wird mit den Bildern einer eigenen Sitzung weiter trainiert (`personal_exp.py`). Der Backbone bleibt dabei eingefroren, auch die BatchNorm-Statistiken.

Weil der Backbone eingefroren ist, werden seine Merkmale nur einmal berechnet und gespeichert (Merkmals-Cache). Danach wird nur der Kopf trainiert. Ein Lauf dauert deshalb nur Sekunden.

3.5. Auswertung

- **Fehlermaß:** Diagonalfehler E_{diag}, also der euklidische RMSE in Prozent der Bildschirmdiagonale (Gleichung (4.1)).
- **Baselines:** Die Konstant-Baseline sagt immer den Mittelwert der Trainingsziele vorher, die Zufalls-Baseline rät zufällig. Beide werden auf denselben Testdaten wie das Modell berechnet (Abschnitt 4.1.6).
- **Statistik:** Bei GazeCapture gibt es fünf Wiederholungen pro Einstellung mit 98 %-Konfidenzintervall (Student-t) und dem Mann-Whitney-U-Test. Bei den Sitzungen nutze ich den Wilcoxon-Vorzeichen-Rang-Test (gepaart) und den Kruskal-Wallis-Test (Vergleich von Gruppen).
- **Aufteilung der Sitzungsdaten:** zufällig 70/15/15, sequenziell (zeitliche Blöcke mit einer Lücke von einer Sekunde) und zwei Videos (Training auf einer Sitzung, Test auf einer anderen) (Abschnitt 4.5).
- **Räumliche Analyse:** Fehler in Mitte, Rändern und Ecken des Bildschirms (Abschnitt 4.3.1).

3.6. Implementierung

Der Code ist in Python mit PyTorch und torchvision geschrieben. Die wichtigsten Teile sind:

- `main.py`: lädt GazeCapture, normiert die Ziele und erstellt die Splits;
- `model.py`, `dataset.py`: Modell, Verluste, Training, Laden der Daten und Merkmals-Cache;
- `metrics.py`: Diagonalfehler und Baselines (eine einzige Quelle für alle Skripte);
- `exp.py`: Hyperparameterstudie mit fünf Wiederholungen;
- `train_base.py`, `personal_exp.py`: Basismodelle und Personalisierung (E0–E3);
- `session_design.py`: Abdeckung, Übergangsbilder und bereinigte Labels;

- `spatial_analysis.py`, `plot_targets.py`: Abbildungen;
- `auswertung.py`: erstellt die Tabellen aus den Ergebnisdateien;
- Notebook `notebooks/Masterarbeit.ipynb`: führt alle Experimente aus und erstellt Tabellen und Abbildungen.

Alle Experimente laufen auf einer CPU. Die Befehle zum Wiederholen der Experimente stehen im Anhang (Anhang A.1).

[**Noch zu schreiben:** Rechner (CPU, Arbeitsspeicher) und Versionen von Python und PyTorch ergänzen.]

4. Experimente und Ergebnisse

Dieses Kapitel beschreibt die Experimente auf dem Datensatz GazeCapture [1] und auf meinen eigenen Aufnahmen. Zuerst erkläre ich Modell, Daten und Auswertung (Abschnitt 4.1). Danach folgen die Trainingseinstellungen (Abschnitt 4.2), die räumliche Verteilung des Fehlers und die Verlustfunktionen (Abschnitt 4.3) und schließlich die Personalisierung (Abschnitte 4.4 bis 4.6). Abschnitt 4.7 fasst zusammen, was die Ergebnisse bedeuten.

Alle Zahlen wurden mit den Protokolldateien der Experimente abgeglichen. Dabei wurde ein Fehler im Auswertungscode gefunden und korrigiert (Abschnitt 4.1.3). Alle Fehlerwerte in diesem Kapitel verwenden die korrigierte Metrik.

4.1. Versuchsaufbau

4.1.1. Modell

- **Backbone:** ResNet-18 [2], auf ImageNet vortrainiert (`ResNet18_Weights.DEFAULT`).
- **Regressionskopf:** vier vollständig verbundene Schichten $512 \to 512 \to 256 \to 128 \to 2$ mit ReLU, Dropout $p = 0{,}2$ nach der zweiten Schicht und einer Aktivierung am Ausgang (Standard: Sigmoid).
- **Optimierer:** AdamW, Weight Decay 10^{-5}.
- **Standard-Verlust:** L1-Verlust (`nn.L1Loss`), siehe Abschnitt 4.3.3.
- **Early Stopping:** Abbruch nach drei Epochen ohne Verbesserung (Patience 3), höchstens 500 Epochen.

4.1.2. Daten und Splits

Die hier verwendete Version von GazeCapture enthält 347 085 gelabelte Bilder von 262 Personen. Ich verwende zwei Splits:

- `norm_subject`: personendisjunkt, das heißt, keine Person ist gleichzeitig in Training und Test;
- `norm_random`: zufällig auf Bildebene, dieselbe Person kann in Training und Test vorkommen.

Für die Hyperparameterstudie ziehe ich 1000 Trainings- und 200 Testbilder *zufällig* aus den Split-Dateien (fester Seed 0, `--sample-seed 0`). Bei `norm_subject` kommen die Trainingsbilder von 179 Personen und die Testbilder von 45 anderen Personen. Ältere Läufe haben einfach die *ersten* Zeilen der sortierten Dateien genommen. Das waren nur fünf Trainingspersonen und eine Testperson. Diese Läufe verwende ich nicht mehr.

4.1.3. Fehlermaß

Alle Blickziele liegen im Einheitsquadrat $[0,1]^2$ (Abschnitt 4.1.5). Der Fehler ist die Wurzel aus dem mittleren quadrierten *euklidischen* Abstand zwischen Vorhersage $\hat{\boldsymbol{p}}_i$ und Ziel $\boldsymbol{p}_i$. Ich gebe ihn in Prozent der Bildschirmdiagonale $\sqrt{2}$ an:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

0 % bedeutet eine perfekte Vorhersage, 100 % einen Fehler so groß wie die ganze Bildschirmdiagonale.

4.1.4. Ablauf der Experimente

Die Hyperparameterstudie startet mit dem Befehl

```
python exp.py --experiment all --sample-seed 0 --val-size 200
```

Ich ändere immer nur einen Wert (zum Beispiel die Batchgröße). Alle anderen Werte bleiben auf der Standardeinstellung (Tabelle 4.1). Jede Einstellung wird fünfmal trainiert. Das Early Stopping nutzt eine eigene Validierungsmenge von 200 Bildern einer weiteren Person. Ich gebe den *Test*fehler in der Epoche mit dem kleinsten Validierungsfehler an. Die Testdaten werden also nie für die Modellauswahl benutzt. Die Tabellen zeigen den mittleren Testfehler, das zweiseitige 98 %-Konfidenzintervall (KI) des Mittelwerts (Student-t, vier Freiheitsgrade) und die mittlere Laufzeit.

Was die „Laufzeit“ misst. Die Laufzeit ist die echte Dauer (Wall-Clock) *eines ganzen Trainingslaufs*, gemittelt über die fünf Läufe. Ein Lauf beginnt mit der Auswertung des untrainierten Modells (Epoche 0) und endet, wenn das Early Stopping abbricht. Er enthält alle Epochen und nach jeder Epoche die Auswertung auf Trainings-, Validierungs- und Testdaten. Die Laufzeit ist *nicht* die Zeit pro Epoche. Sie enthält *nicht* das Laden der Bilder und nicht die einmalige Berechnung der Backbone-Merkmale vor dem Lauf. Weil das Early Stopping verschieden früh abbricht, zeigt die Zeit auch, wie lange sich eine Einstellung noch verbessert.

Tabelle 4.1.: Standardeinstellung von `exp.py`. Jede Variation ändert genau einen dieser Werte.

Parameter	Wert	Parameter	Wert
Split	`norm_subject`	Trainierbare Schichten	nur FC
Trainings-/Testgröße	1000 / 200[a]	Ausgabe-Aktivierung	Sigmoid
Batchgröße	32	Verlust	L1 (`standard`)
Lernrate	10^{-4}	Max. Epochen / Patience	500 / 3
Weight Decay	10^{-5}	Validierungsmenge	200 Bilder[b]
Wiederholungen	5		

[a] zufällig gezogen, Seed 0. [b] von einer Person, die weder im Training noch im Test vorkommt.

So liest man die Tabellen. In jeder Hyperparametertabelle ist die gewählte Option mit † markiert. Die Spalte „p vs. †“ enthält den p-Wert eines zweiseitigen Mann-Whitney-U-Tests zwischen den fünf Läufen dieser Option und den fünf Läufen der gewählten Option. Ich verwende einen Rangtest, weil man mit fünf Läufen nicht prüfen kann, ob die Werte normalverteilt sind, und weil einige Optionen einzelne Ausreißer haben (zum Beispiel ein ReLU-Lauf mit 46,98 %). Bei 5 + 5 Läufen ist der kleinste mögliche p-Wert 0,008. Er bedeutet: *jeder* Lauf der einen Option ist besser als *jeder* Lauf der anderen. Ein Mittelwert ist nur dann **fett**, wenn die gewählte Option *signifikant besser* ist als jede andere Option der Tabelle ($p < 0{,}05$ und kleinerer Mittelwert).

Drei Punkte sind dabei wichtig:

1. **Gleiche Daten in allen Wiederholungen.** Der Seed legt Trainings-, Test- und Validierungsdaten fest. Die fünf Wiederholungen unterscheiden sich nur in den Startgewichten, der Reihenfolge der Daten und im Dropout. Das KI zeigt also die Zufälligkeit des Trainings, nicht den Einfluss anderer Datenteilmengen.
2. **Kleine Validierungsmenge.** Die 200 Validierungsbilder stammen von nur einer Person. Das Early Stopping ist daher etwas zufällig. Die Testdaten werden aber nie für die Auswahl benutzt.
3. **Trainingszeit.** Bei „nur FC“ wird der Kopf auf gespeicherten ResNet-Merkmalen trainiert (512 Zahlen pro Bild statt des Bildes). Ein Lauf dauert daher nur etwa eine Sekunde, und Unterschiede in diesen Zeiten sind unwichtig. Bei FC + Layer 4 und beim vollen ResNet läuft in jeder Epoche jedes Bild vorwärts und rückwärts durch den Backbone. Diese Zeiten sind viel länger und nicht direkt mit „nur FC“ vergleichbar.

4.1.5. Normierung der Zielwerte

Die rohen Blickziele in GazeCapture sind Bildschirmpunkte des jeweiligen Geräts, gemessen von der oberen linken Ecke. Ihr Wertebereich hängt daher von Bildschirmgröße und Ausrichtung ab. Die 347 085 Bilder der 262 Personen stammen von zehn Bildschirmformaten,

von 320×480 bis 1024×768 Punkten, im Hoch- und im Querformat. Ich teile die Ziele deshalb durch die Bildschirmbreite W und -höhe H des jeweiligen Bildes (`main.py`; Werte außerhalb von [0,1] werden abgeschnitten, das betrifft 0,006 % der Bilder):

$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

Abbildung 4.1 zeigt die Wirkung. Ohne Normierung liegen die Ziele der vielen Smartphone-Bilder vor allem oben links, und dieselbe Koordinate bedeutet auf jedem Gerät eine andere Stelle (Abbildung 4.1a). Die Fehlerwerte sind dann zwischen Geräten nicht vergleichbar (linke Spalte). Nach der Normierung liegen alle Ziele in $[0,1]^2$, und Gleichung (4.1) ist sinnvoll definiert (rechte Spalte). Alle weiteren Experimente verwenden normierte Ziele.

Warum beginnt die Kurve bei der Konstant-Baseline und nicht bei der Zufalls-Baseline? In Abbildung 4.1b beginnen die Kurven in Epoche 0, also *vor* dem ersten Trainingsschritt, bei 26,77 %. Man könnte erwarten, dass ein untrainiertes Modell so schlecht ist wie zufälliges Raten (etwa 40 %). Das stimmt aber nicht.

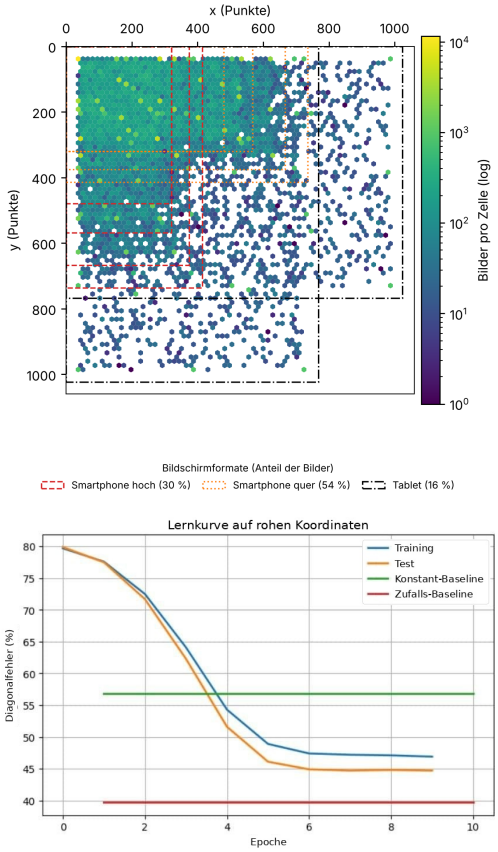
Die Zufalls-Baseline rät für jedes Bild einen *anderen* zufälligen Punkt auf dem Bildschirm. Ein untrainiertes Netz macht das nicht. Seine Gewichte sind kleine Zufallszahlen. Die letzte lineare Schicht gibt daher für jedes Bild Werte nahe 0 aus, und die Sigmoid macht daraus $\sigma(0) = 0{,}5$. Das untrainierte Modell sagt also für alle Bilder fast *denselben* Punkt vorher: die Bildschirmmitte (0,5; 0,5). Genau das macht die Konstant-Baseline. Richtig ist daher:

- Epoche 0 $\approx$ Konstant-Baseline *derselben* Testdaten („immer in die Mitte schauen“);
- erst wenn das Training die Kurve *unter* diesen Wert bringt, hat das Modell etwas aus den Bildern gelernt;
- die Zufalls-Baseline ist eine obere Grenze für „reines Raten“; ein Modell sollte nie darüber liegen.

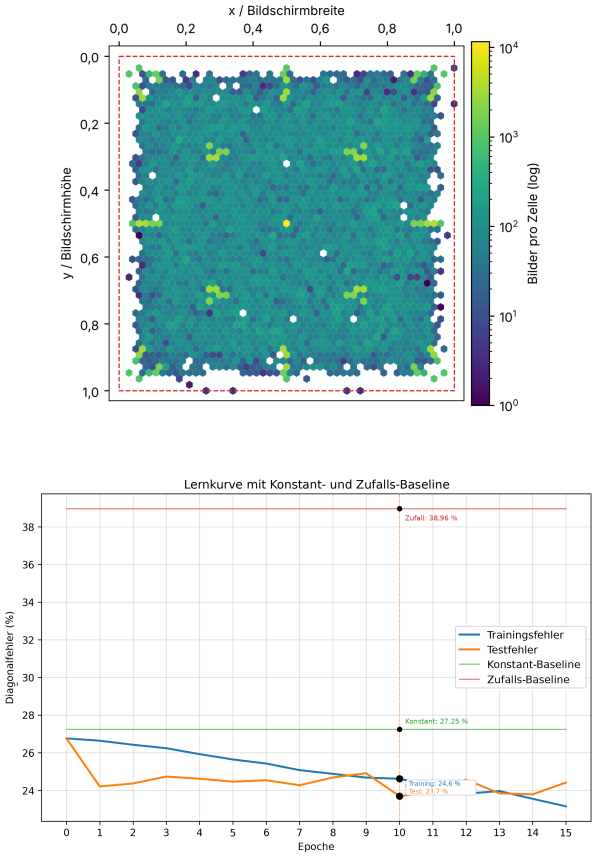
Die Messwerte bestätigen das: Auf den 200 Testbildern dieses Laufs liegt die Konstant-Baseline bei 26,76 % (Mittelwert der Trainingsziele) bzw. 26,71 % (Bildschirmmitte). Das entspricht dem Startwert von 26,77 %. Wichtig ist, die Baseline auf *denselben* Daten wie die Kurve zu berechnen. Eine Baseline auf dem gesamten Datensatz (27,25 %) liegt etwas höher, und die Kurve scheint dann „unter der Baseline“ zu beginnen. Der Code berechnet die Baselines deshalb jetzt auf derselben Teilmenge (`metrics.subset_baselines`).

4.1.6. Baselines

Ich verwende zwei einfache Vergleichsschätzer. Die *Konstant-Baseline* sagt für jedes Testbild den Mittelwert der Trainingsziele vorher. Die *Zufalls-Baseline* zieht Vorhersagen gleichverteilt aus $[0,1]^2$. Ein Modell ist nur dann nützlich, wenn es deutlich besser ist als die Konstant-Baseline.



(a) Rohe Ziele in Bildschirmpunkten (Dichte aller Bilder, logarithmische Farbskala); gestrichelt: Umrisse der Bildschirmformate. Unten: Lernkurve eines frühen Laufs auf rohen Koordinaten; diese Werte sind keine Prozente einer gemeinsamen Diagonale und nicht mit den anderen Abbildungen vergleichbar.



(b) Normierte Ziele (gleiche Darstellung; gestrichelt: Einheitsquadrat). Die hellen Zellen sind Kalibrierpunkte, die auf allen Geräten an denselben relativen Stellen gezeigt werden. Unten: Lernkurve auf `norm_random` (1000/200) mit dem Fehlermaß aus Gleichung (4.1). Epoche 0 ist das untrainierte Modell. Die waagerechten Linien sind Baselines auf denselben Daten wie die Kurven.

Abbildung 4.1.: Rohe und normierte Blickziele im Vergleich.

Tabelle 4.2.: Baseline-Fehler E_{diag} in %. Oben: gesamter Datensatz (`exp.py`, Bootstrap mit 30 Stichproben zu je 2000 Bildern, Mittelwert und 98 %-KI). Unten: dieselben zufälligen Trainings-/Testteilmengen (Seed 0) wie beim Modelltraining (kein KI, eine feste Teilmenge).

Daten	Konstant-Baseline	Zufalls-Baseline
Gesamter Datensatz (347 085 Bilder)	27,28 [27,15; 27,40]	39,82 [39,60; 40,04]
`norm_subject`, 1000/200	28,46	40,36
`norm_subject`, 10 000/2000	27,62	39,96
`norm_random`, 1000/200	27,66	39,83

Der untere Teil von Tabelle 4.2 ist die richtige Referenz für alle Modellergebnisse in Abschnitt 4.2, weil er auf denselben Daten berechnet wurde.

4.2. Trainingseinstellungen

Alle Ergebnisse dieses Abschnitts stammen aus einem Lauf (`exp_results_all.json`; `exp.py`, korrigierte Metrik, zufällige Teilmengen mit Seed 0, Early Stopping auf der Validierung, fünf Wiederholungen pro Einstellung). Die Konstant-Baseline der Standardeinstellung auf denselben Daten liegt bei 28,46 %.

4.2.1. Batchgröße

Tabelle 4.3.: Batchgröße. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines ganzen Trainingslaufs (Kopf auf gespeicherten Merkmalen, siehe Abschnitt 4.1.4).

Batchgröße	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Laufzeit (min)
32†	28,39	[28,21; 28,58]	—	0,019
64	28,35	[28,29; 28,41]	0,69	0,012
128	28,38	[28,30; 28,47]	0,84	0,009

Die drei Batchgrößen ergeben denselben Testfehler (Tabelle 4.3): Die Mittelwerte unterscheiden sich um höchstens 0,05 Prozentpunkte, die Konfidenzintervalle überlappen, und der Rangtest findet keinen Unterschied ($p \geq 0{,}69$). Ein Lauf dauert weniger als eine Sekunde, weil nur der kleine Kopf auf gespeicherten Merkmalen trainiert wird. Die Zeitunterschiede sind daher unwichtig.

Ich behalte die Batchgröße 32 als Standard. Das Experiment zeigt keinen Nachteil, und zwei praktische Gründe sprechen für eine kleine Batchgröße bei der Personalisierung: (i) Eine persönliche Sitzung hat nur einige hundert Trainingsbilder. Mit kleinen Batches gibt es mehr Updates pro Epoche. (ii) Das Rauschen kleiner Batches wirkt nach gängiger Meinung leicht regularisierend. Die Daten dieses Experiments *beweisen* aber keinen Vorteil von 32 gegenüber 64 oder 128.

4.2.2. Lernrate

Auch bei der Lernrate gibt es keinen signifikanten Unterschied (Tabelle 4.4): Alle Mittelwerte liegen zwischen 28,33 % und 28,43 %, die Intervalle überlappen, und kein Rangtest ist signifikant ($p \geq 0{,}056$). Der Endfehler allein kann die Wahl also nicht begründen. Ich wähle $\eta = 10^{-4}$ wegen des *Lernverlaufs* in Abbildung 4.2:

Tabelle 4.4.: Lernrate. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines ganzen Trainingslaufs.

Lernrate	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Laufzeit (min)
10^{-3}	28,43	[28,33; 28,54]	0,15	0,018
$10^{-4\dagger}$	28,37	[28,32; 28,42]	—	0,020
10^{-5}	28,33	[28,28; 28,38]	0,056	0,022
10^{-6}	28,37	[28,21; 28,52]	0,42	0,055

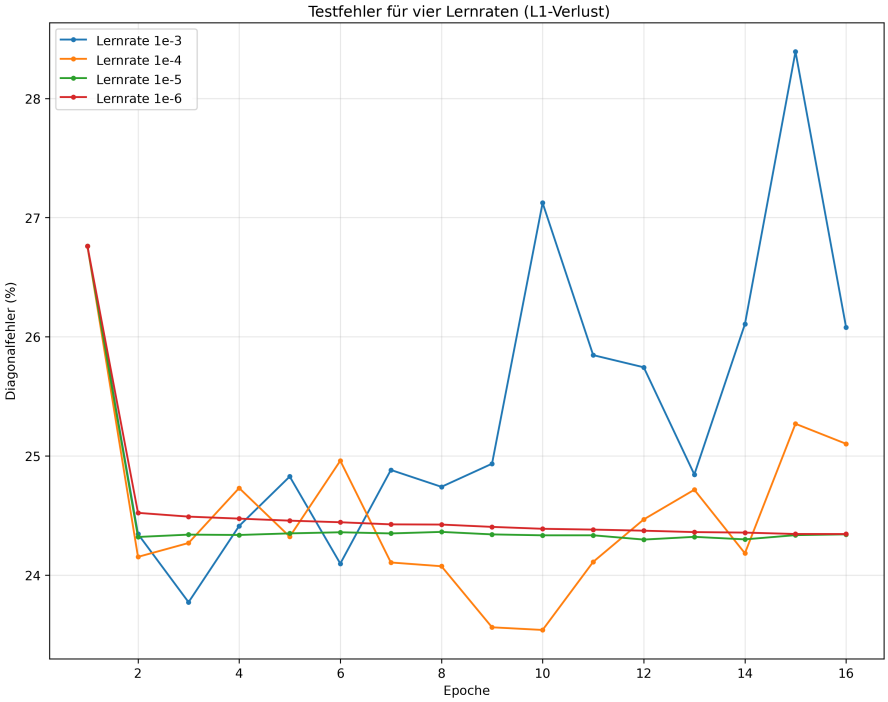


Abbildung 4.2.: Testfehler einzelner Trainingsläufe mit vier Lernraten (`norm_random`, 1000/200, 15 Epochen ohne Early Stopping, Fehlermaß aus Gleichung (4.1)). Der erste Punkt (Epoche 1 auf der Achse) ist das untrainierte Modell.

- $\eta = 10^{-3}$: Die Schritte sind zu groß. Nach einem schnellen Abfall schwankt der Testfehler stark und steigt wieder (Spitzen bei etwa 27,1 % und 28,4 %). Der Lauf findet kein stabiles Minimum.
- $\eta = 10^{-4}$: erreicht den kleinsten Testfehler aller Kurven (etwa 23,6 % um Epoche 10). Die Kurve schwankt noch, aber viel weniger als bei 10^{-3}. Das Early Stopping kann den besten Stand behalten.
- $\eta = 10^{-5}$ **und** 10^{-6}: Die Kurven sind sehr glatt und fallen gleichmäßig, bleiben aber bei etwa 24,3 % stehen. Die Schritte sind zu klein, um in der verfügbaren Zeit weiterzukommen. Das Ergebnis ist stabil, aber nicht das beste.

Die Trainingszeit ist hier *kein* Kriterium: Sie liegt für alle Lernraten unter einer Sekunde (gespeicherte Merkmale). Zu beachten ist, dass Abbildung 4.2 einzelne Läufe zeigt. Die Wiederholung mit fünf Läufen in Tabelle 4.4 bestätigt keinen signifikanten Unterschied.

4.2.3. Trainierbare Schichten

Ich vergleiche drei Tiefen des Fine-Tunings (Tabelle 4.5): nur den Regressionskopf (nur FC), den Kopf plus die letzte Residualstufe (`layer4`) und das ganze Netz.

Tabelle 4.5.: Trainierbare Parameter der drei Varianten (ResNet-18 mit Regressionskopf, insgesamt 11 603 650 Parameter).

	Nur FC	FC + Layer 4	Volles ResNet
Trainierbare Parameter	427 138	8 820 866	11 603 650
Anteil an allen Parametern	3,68 %	76,02 %	100,00 %
Backbone-Merkmale speicherbar	ja	nein	nein

Tabelle 4.6.: Trainierbare Schichten. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines ganzen Trainingslaufs. Bei „nur FC“ läuft kein Bild durch den Backbone (gespeicherte Merkmale).

Trainierbare Schichten	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Laufzeit (min)
Nur FC†	28,36	[28,23; 28,49]	—	0,015
FC + Layer 4	29,07	[26,58; 31,55]	0,22	9,160
Volles ResNet	29,39	[26,99; 31,78]	0,31	15,366

„Nur FC“ hat den kleinsten mittleren Fehler und mit Abstand das schmalste Intervall (Tabelle 4.6). Mehr trainierbare Schichten helfen nicht. Sie machen das Ergebnis sogar *unsicherer*: Ein Lauf mit FC + Layer 4 (31,72 %) und zwei Läufe mit dem vollen ResNet (30,70 % und 31,18 %) enden deutlich über der Konstant-Baseline (28,46 %). Alle Läufe mit „nur FC“ bleiben zwischen 28,28 % und 28,47 %. Mit nur fünf Läufen sind die Unterschiede nicht signifikant (p = 0,22 und 0,31); der Wert für „nur FC“ ist daher nicht fett. Die

Trainingszeit wächst stark mit der Zahl der Parameter: 9,2 min pro Lauf für FC + Layer 4 und 15,4 min für das ganze Netz.

Ich wähle „nur FC“ aus drei Gründen:

1. **Nicht schlechter, aber stabiler:** Mehr Schichten senken den Fehler hier nicht, sie erhöhen nur die Streuung zwischen den Läufen.
2. **Robust bei wenigen Daten:** Eine persönliche Sitzung hat nur einige hundert Bilder. Wenn man Millionen von Backbone-Parametern mit so wenigen Beispielen trainiert, droht Überanpassung, und die vortrainierten Merkmale werden verzerrt. Kumar u. a. [22] zeigen, dass vollständiges Fine-Tuning vortrainierte Merkmale beschädigen und schlechter generalisieren kann als reines Kopftraining.
3. **Aufwand:** Nur 3,68 % der Parameter werden trainiert, und die Backbone-Merkmale werden nur einmal berechnet. Die Personalisierung ist dadurch schnell und braucht wenig Speicher.

4.2.4. Aktivierung am Ausgang

Die Aktivierung am Ausgang kommt nach der letzten linearen Schicht, die (x,y) ausgibt. Sigmoid bildet auf (0,1) ab, also genau auf den Bereich der normierten Koordinaten. Die Gauß-Aktivierung $g(z) = \exp(-z^2/2\sigma^2)$ mit $\sigma = 1$ bildet auf (0,1] ab, ist aber nicht monoton. ReLU entfernt nur negative Werte, und „Keine“ lässt die Ausgabe unverändert.

Tabelle 4.7.: Aktivierung am Ausgang. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und mittlere Dauer eines ganzen Trainingslaufs.

Aktivierung	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Laufzeit (min)
Sigmoid†	**28,36**	[28,29; 28,43]	—	0,022
Gauß	31,29	[28,44; 34,14]	0,008	0,019
ReLU	32,90	[19,69; 46,10]	0,008	0,028
Keine	29,39	[28,42; 30,36]	0,008	0,032

Sigmoid ist der einzige klare Gewinner der Hyperparameterstudie (Tabelle 4.7): kleinster Fehler, schmalstes Intervall, und *jeder* Sigmoid-Lauf ist besser als *jeder* Lauf der drei anderen Aktivierungen ($p = 0{,}008$ in allen drei Vergleichen). Der Wert ist daher fett. Die anderen Aktivierungen sind auch weniger stabil: Ein ReLU-Lauf endet bei 46,98 %, also schlechter als zufälliges Raten. Ein möglicher Grund: ReLU kann für alle Eingaben null ausgeben („tote“ Ausgabe), und der Gradient erholt sich davon nicht mehr. Ohne begrenzende Aktivierung (Keine, ReLU) kann das Netz außerdem Punkte außerhalb des Bildschirms vorhersagen. Ich wähle Sigmoid, weil sie die Ausgabe auf den Bildschirm begrenzt und den kleinsten und stabilsten Fehler ergibt.

4.2.5. Datenmenge

Tabelle 4.8.: Datenmenge (Training/Test). Mittlerer Testfehler über fünf Läufe mit 98 %-KI, mittlere Dauer eines ganzen Trainingslaufs und Konstant-Baseline der jeweiligen Testdaten.

Training / Test	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Laufzeit (min)	Konst.-Baseline (%)
10 000 / 2000	27,64	[27,55; 27,73]	0,008	0,167	27,62
1000 / 200†	28,39	[28,33; 28,44]	—	0,018	28,46

Die größere Datenmenge hat einen kleineren mittleren Fehler (Tabelle 4.8). Die beiden Varianten werden aber auf *verschiedenen* Testdaten ausgewertet, und jeder Mittelwert ist praktisch gleich der Konstant-Baseline seiner eigenen Testdaten (27,64 % vs. 27,62 % und 28,39 % vs. 28,46 %). Der Unterschied von 0,75 Prozentpunkten kommt also von den unterschiedlichen Testdaten und nicht von mehr Trainingsdaten – auch wenn der Rangtest signifikant ist ($p = 0{,}008$). Ich behalte die kleine Variante (1000/200), weil sie etwa zehnmal schneller ist und etwa so viele Daten hat wie eine Personalisierungssitzung.

4.2.6. Art des Splits

Tabelle 4.9.: Art des Splits. Mittlerer Testfehler über fünf Läufe mit 98 %-KI und Konstant-Baseline der jeweiligen Testdaten.

Split	Mittl. Testfehler (%)	98 %-KI (%)	p vs. †	Konstant-Baseline (%)
`norm_random`	27,72	[27,64; 27,80]	0,008	27,66
`norm_subject`†	28,39	[28,26; 28,53]	—	28,46

Der zufällige Split ergibt den kleineren Fehler (Tabelle 4.9). Auch hier kommt der größte Teil des Unterschieds von den Testdaten: Die Konstant-Baseline ist auf `norm_random` schon um 0,80 Prozentpunkte niedriger, und beide Modelle liegen höchstens 0,07 Prozentpunkte neben ihrer Baseline. Außerdem können beim zufälligen Split dieselben Personen in Training und Test vorkommen. Das macht die Aufgabe leichter und das Ergebnis zu optimistisch. Ich verwende daher `norm_subject` als Standard. Nur dieser Split misst die Leistung auf unbekannten Personen, also die Situation vor der Personalisierung.

4.3. Räumliche Fehleranalyse und Verlustfunktionen

Der mittlere Fehler aus Abschnitt 4.2 zeigt nicht, *wo* auf dem Bildschirm die Fehler entstehen. Dieser Abschnitt untersucht zuerst die räumliche Verteilung des Fehlers (Ab-

schnitte 4.3.1 und 4.3.2). Das Ergebnis führte zur Idee, den Verlust nach der Position zu gewichten (Abschnitt 4.3.3). Abschnitt 4.3.4 wertet diese Idee aus.

4.3.1. Einteilung des Bildschirms

Ich beschreibe den normierten Bildschirm $[0,1]^2$ mit einem 3×3-Raster aus drei Regionen:

Mitte:	(0,50; 0,50),
Ränder:	(0,10; 0,50), (0,50; 0,10), (0,90; 0,50), (0,50; 0,90),
Ecken:	(0,10; 0,10), (0,10; 0,90), (0,90; 0,10), (0,90; 0,90).

Ein Testbeispiel mit Ziel $\boldsymbol{p}$ gehört zum Rasterpunkt $\boldsymbol{g}$, wenn $\|\boldsymbol{p}-\boldsymbol{g}\|_2 \le r$ mit $r = 0{,}15$ gilt. Der Fehler eines Rasterpunkts ist der mittlere euklidische Fehler $\|\hat{\boldsymbol{p}}-\boldsymbol{p}\|_2$ seiner Beispiele (in normierten Einheiten, also als Anteil der Bildschirmseite; die Diagonale ist $\sqrt{2} \approx 1{,}41$). Der Fehler einer Region ist der Mittelwert über alle Beispiele ihrer Rasterpunkte. Diese Definitionen gelten für alle Abbildungen und Tabellen in diesem Abschnitt.

4.3.2. Fehler-Heatmaps

Abbildung 4.3 nutzt die 200 Testvorhersagen des Standardmodells (`norm_subject`, L1-Verlust; Datei `results/heatmap/base_loss1.csv`). Für eine Position $\boldsymbol{c}$ (Mitte, Rand oder Ecke) berechne ich den mittleren Fehler $\bar{e}(\boldsymbol{c})$ aller Beispiele im Radius r und zeichne ihn als Gaußfleck:

$$H(x,y) = \bar{e}(\boldsymbol{c})\,\exp\!\left(-\frac{\|(x,y)-\boldsymbol{c}\|_2^2}{2\sigma^2}\right), \qquad \sigma = 0{,}20. \tag{4.3}$$

Die Farbe zeigt also die *Größe* des lokalen Fehlers, und die Lage des hellen Flecks zeigt, *wo* er gemessen wurde. Alle drei Teilbilder nutzen dieselbe Farbskala (von 0 bis zum größten Wert), damit man die Farben direkt vergleichen kann.

Der Fehler wächst von innen nach außen: Er beträgt 0,086 in der Mitte ($n = 18$), 0,432 an den Rändern ($n = 62$) und 0,586 in den Ecken ($n = 42$). Die Randfehler sind etwa fünfmal und die Eckfehler etwa siebenmal so groß wie in der Mitte.

Das vierte Teilbild erklärt den Grund. Alle Vorhersagen liegen in einem sehr kleinen Bereich um (0,5; 0,5). Ihre Standardabweichung beträgt in beiden Koordinaten nur etwa 0,004, die der Ziele dagegen etwa 0,28. Das Modell sagt also für jedes Bild fast denselben Punkt vorher. Sein Fehler ist einfach der Abstand des Ziels von der Bildschirmmitte. Ein Schätzer, der immer die Mitte ausgibt, liefert praktisch dieselben Werte (Mitte 0,085, Ränder 0,432, Ecken 0,587; insgesamt $E_{\text{diag}} = 28{,}45\,\%$ gegenüber $28{,}47\,\%$ für das Modell).

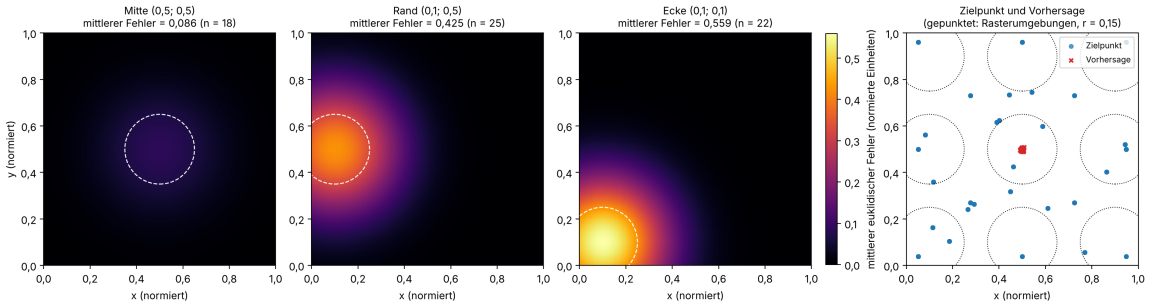


Abbildung 4.3.: Räumlicher Fehler des Standardmodells (L1-Verlust, `norm_subject`, 200 Testbeispiele). Teilbilder 1–3: Fehler-Heatmaps ($r = 0{,}15$, $\sigma = 0{,}20$, gemeinsame Farbskala) in der Mitte (0,5; 0,5), am linken Rand (0,1; 0,5) und in der unteren linken Ecke (0,1; 0,1). Der Fehler wächst von der Mitte (0,086) über den Rand (0,425) zur Ecke (0,559). Teilbild 4: Ziele (blau) und Vorhersagen (rot); alle Vorhersagen liegen nahe der Bildschirmmitte.

4.3.3. Verlustfunktionen

Wegen der großen Fehler an Rändern und Ecken entstand die Idee, diese Bereiche im Verlust stärker zu gewichten. Sei $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ die Vorhersage und $\boldsymbol{p}_i = (x_i, y_i)$ das Ziel von Beispiel i in einem Batch der Größe N.

L1-Verlust (`nn.L1Loss`, „standard“).

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} \big(|\hat{x}_i - x_i| + |\hat{y}_i - y_i| \big). \tag{4.4}$$

Der Gradient hat immer denselben Betrag, egal wie groß der Fehler ist. Der Verlust ist daher robust gegen Ausreißer (zum Beispiel falsche Labels beim Zielwechsel), behandelt aber alle Bildschirmpositionen gleich.

Smooth-L1-Verlust (`nn.SmoothL1Loss`, $\beta = 1$). Für ein Residuum u gilt:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{sonst.} \end{cases} \tag{4.5}$$

Er ist quadratisch für kleine und linear für große Residuen. Da hier alle Residuen kleiner als $\beta = 1$ sind, verhält er sich wie ein skalierter quadratischer Fehler: Bei kleinen Fehlern wird der Gradient klein.

Sensitiver L1-Verlust (nach Position gewichtet, `sensitive`). Jedes Beispiel wird mit seinem L_1-Abstand zur Bildschirmmitte gewichtet:

$$d_i = |x_i - 0{,}5| + |y_i - 0{,}5| \in [0{,}1], \qquad w_i = 1 + d_i \in [1{,}2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

Ein Beispiel in der Ecke zählt doppelt so viel wie eines in der Mitte, ein Beispiel am Rand etwa 1,4-mal so viel. Der Gradient für äußere Ziele wird größer, und die Vorhersagen sollen dadurch von der Mitte weggezogen werden. (Diese Formel entspricht der Implementierung in `model.py`.)

Der „verbesserte sensitive Verlust“ in Tabelle 4.11 war eine weitere Variante aus einem Notebook zum Ausprobieren. Seine genaue Formel ist im aktuellen Code nicht mehr enthalten. Ich nenne ihn nur der Vollständigkeit halber.

4.3.4. Ergebnisse des Verlustvergleichs

Tabelle 4.10 zeigt den globalen Vergleich mit fünf Läufen und dazu die Fehler in den Bildschirmregionen aus Abschnitt 4.3.1.

Tabelle 4.10.: Vergleich von L1- und sensitivem L1-Verlust. Links: mittlerer Testfehler E_{diag} über fünf Läufe (`exp.py`, 98 %-KI) und mittlere Laufzeit. Rechts: mittlerer euklidischer Fehler pro Bildschirmregion (normierte Einheiten; je ein Lauf, 200 Testbeispiele, $r = 0{,}15$; n = Anzahl der Beispiele). Letzte Zeile: Schätzer, der immer die Mitte ausgibt, auf denselben 200 Testbeispielen.

	Global (5 Läufe)			Räumlicher Fehler (1 Lauf)		
Verlust	E_{diag} (%)	98 %-KI (%)	Laufzeit (min)	Mitte ($n = 18$)	Ränder ($n = 62$)	Ecken ($n = 42$)
L1 (standard)†	**28,32**	[28,23; 28,42]	0,011	0,086	0,432	0,586
Sensitiver L1	28,48	[28,19; 28,78]	0,021	0,090	0,431	0,585
Konstante Mitte	28,45[a]	—	—	0,085	0,432	0,587

[a] Auf denselben 200 Testbeispielen wie die räumliche Analyse.

Beobachtungen.

- Global ist L1 leicht, aber signifikant besser als der sensitive Verlust: 28,32 % gegenüber 28,48 % (Mann-Whitney $p = 0{,}032$, Tabelle 4.10); der L1-Wert ist daher fett. Der sensitive Verlust streut außerdem stärker zwischen den Läufen. Beide Werte liegen nahe an der Konstant-Baseline (28,46 %).

Tabelle 4.11.: Zusatzexperiment: räumlicher Fehler von vier Verlustfunktionen nach 15 Epochen auf Trainingsdaten mit 52 Eckbeispielen (zwei Läufe, A und B, Werte wie protokolliert). Die Zahlen stammen aus einem Notebook zum Ausprobieren. Die Berechnung der Regionswerte ist im aktuellen Code nicht enthalten; sie sind daher nicht direkt mit Tabelle 4.10 vergleichbar.

	Gesamt (%)		Mitte (%)		Ränder (%)		Ecken (%)	
Verlust	A	B	A	B	A	B	A	B
L1 (`nn.L1Loss`)	24,54	24,63	17,01	16,74	28,28	28,98	34,93	35,07
Smooth L1 (`nn.SmoothL1Loss`)	23,94	24,91	15,40	16,41	29,78	30,59	34,13	35,17
Sensitiver L1	25,57	25,55	19,11	18,61	29,87	29,93	33,41	34,20
Verbesserter sensitiver L1	25,53	27,75	19,20	21,86	29,17	35,03	33,77	31,58

- Räumlich ändert die Gewichtung die Regionsfehler um weniger als 0,005 normierte Einheiten. Rand- und Eckfehler bleiben etwa fünf- bis siebenmal so groß wie in der Mitte.
- Beide Modelle haben dasselbe räumliche Muster wie ein Schätzer, der immer die Mitte ausgibt. Die Gewichtung vergrößert zwar den Gradienten für äußere Beispiele, aber das reicht nicht, damit das Modell die Mitte verlässt.
- Im Zusatzexperiment (Tabelle 4.11) senken die sensitiven Varianten den Eckfehler etwas (sensitiver L1 um 0,9–1,5, die verbesserte Variante um 1,2–3,5 Prozentpunkte). Sie erhöhen aber den Fehler in der Mitte um 1,9–5,1 Prozentpunkte und den Gesamtfehler in allen Läufen. Die beiden Läufe unterscheiden sich stark; diese Tendenzen sind daher nicht verlässlich.

Fazit. Die räumliche Analyse zeigt, *was* das Problem ist: Die Fehler liegen vor allem an den Rändern und in den Ecken. Der gewichtete Verlust sollte diese Fehler senken. Er hat die Ecken aber nicht verbessert und war global sogar leicht, aber signifikant schlechter als L1 ($p = 0{,}032$). Im Zusatzexperiment erhöhte er den Fehler in der Mitte. Ich behalte daher den normalen L1-Verlust: Er hat den kleineren Fehler, ist einfacher, hat keinen zusätzlichen Parameter, und sein konstanter Gradient macht ihn robust gegen falsche Labels in den eigenen Aufnahmen. Die Analyse zeigt auch, *warum* die Gewichtung nicht helfen konnte: Die großen Fehler außen entstehen nicht durch eine falsche Gewichtung, sondern weil das Modell die Bildschirmmitte kaum verlässt (Abschnitt 4.7).

4.4. Personalisierungssitzungen

4.4.1. Aufbau

In jeder persönlichen Sitzung wird der Kopf eines GazeCapture-Basismodells mit den Bildern einer eigenen Aufnahme neu trainiert (nur FC, Sigmoid, L1, $\eta = 10^{-4}$, Weight Decay 10^{-5}, Batchgröße 32, AdamW). Wenn nichts anderes steht, wird jede Sitzung zufällig in 70 % Training, 15 % Validierung und 15 % Test geteilt (Seed 42). Das Early Stopping (Patience 3) nutzt den Validierungsfehler. Ich gebe den Testfehler in der Epoche mit dem kleinsten Validierungsfehler an. Ist die beste Epoche 0, hat die Personalisierung die Validierung nicht verbessert. Der angegebene Testfehler ist dann der des unveränderten Basismodells.

Alle Ergebnisse ab hier stammen aus dem Skript `personal_exp.py` (Notebook `notebooks/Masterarbeit.ipynb`) und verwenden die korrigierte Metrik (Gleichung (4.1)). Alle Zufalls-Seeds sind fest. Der Backbone ist einschließlich seiner BatchNorm-Statistiken vollständig eingefroren.

Merkmals-Cache. Weil der Backbone eingefroren ist, liefert er für ein Bild in jeder Epoche dieselben 512 Merkmale. Ich berechne diese Merkmale deshalb nur einmal pro Sitzung und Basismodell (ohne Augmentierung) und speichere sie. Danach wird nur noch der Kopf trainiert. Das Ergebnis ändert sich dadurch nicht, aber ein Lauf dauert auf einer CPU nur Sekunden. Der einzige Unterschied zu älteren Notebook-Läufen: Bei der Personalisierung gibt es keine Bildaugmentierung.

Basismodelle (E0). Die beiden Basismodelle werden mit der Einstellung aus Abschnitt 4.2 (nur FC, Sigmoid, L1, $\eta = 10^{-4}$, Batchgröße 32, 1000/200 zufällige Bilder, Seed 0, Early Stopping auf der Validierung) auf `norm_subject` und `norm_random` trainiert. Von fünf Seeds behalte ich jeweils den Lauf mit dem kleinsten *Validierungs*fehler (`train_base.py`, `models/base/base_<split>.pt`; eine JSON-Datei dokumentiert Einstellung und alle Seeds). Für `norm_subject` ist das Seed 0 (Validierung 27,37 %, Test 28,70 %; Konstant-Baseline derselben Testdaten 28,46 %), für `norm_random` Seed 3 (Validierung 26,74 %, Test 27,81 %; Konstant-Baseline 27,66 %). Die fünf Seeds unterscheiden sich um weniger als 0,4 Prozentpunkte; die Wahl des Seeds ist also unkritisch. Wie in Abschnitt 4.7 beschrieben, liegen beide Basismodelle auf dem Niveau der Konstant-Baseline.

Von 27 aufgenommenen Sitzungen verwende ich 18: Gen 1 (01–04), Gen 2 (05–09) und Gen 4 (19–27). Gen 3 (10–18) habe ich verworfen, weil das Modell daraus nichts lernen konnte. Tabelle 4.12 zeigt die Aufnahmebedingungen.

Tabelle 4.12.: Aufnahmebedingungen der 18 Sitzungen. Gen 1: Ziel bewegt sich gleichmäßig, ohne Sprünge; Gen 2: Ziel mit zufälligen Sprüngen; Gen 4: 5-s-Abschnitte (0,5 s WAIT an einem neuen Startpunkt, 4,5 s MOVE), 90 % der Startpunkte in den Ecken. Dauer und Betriebssystem wurden nur in Gen 4 verändert (Gen 1/2: Ubuntu, Dauer nicht protokolliert).

Sitzungen	Generation	Brille	Licht / Hintergrund
01, 03, 04	Gen 1	mit	01, 04: hell; 03: dunkel
02	Gen 1	ohne	hell
05, 07, 08	Gen 2	mit	05: hell; 07: dunkel; 08: sehr dunkel
06, 09	Gen 2	ohne	06: hell; 09: sehr dunkel
Gen 4	(Dauer / BS)		
19, 20	Gen 4	mit	ruhig; 120 s / 90 s; Ubuntu
21, 22	Gen 4	ohne	ruhig; 120 s / 90 s; Ubuntu
23 / 24	Gen 4	mit / ohne	unruhig; 120 s; Ubuntu
25 / 26	Gen 4	mit / ohne	unruhig; 120 s; Windows
27	Gen 4	mit	unruhig; 90 s; Windows

4.4.2. Ergebnisse aller Sitzungen (E1)

Tabelle 4.13 zeigt jede Sitzung mit beiden Basismodellen.

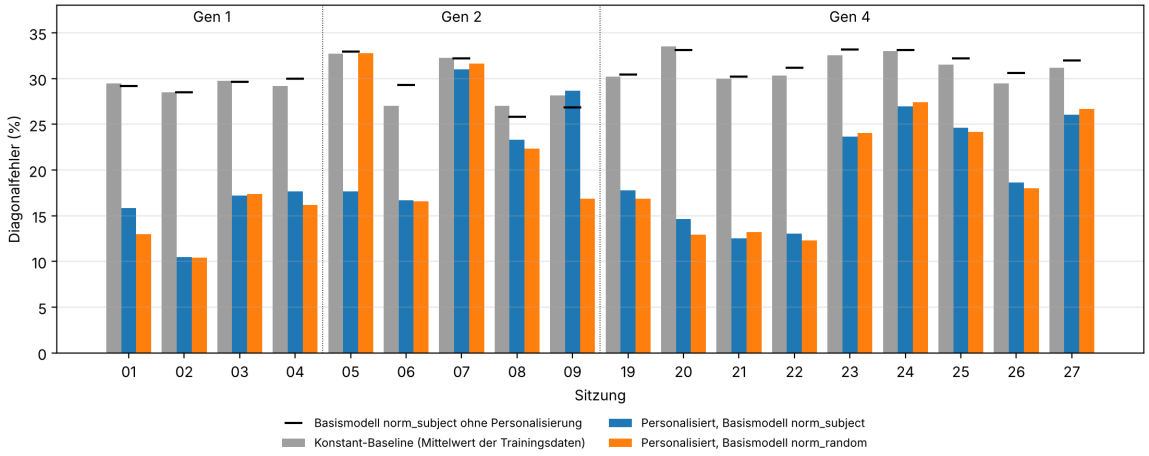


Abbildung 4.4.: Testfehler der 18 Sitzungen nach der Personalisierung mit beiden Basismodellen, verglichen mit der Konstant-Baseline und dem unveränderten Basismodell (`norm_subject`) auf denselben Testbildern (Daten aus Tabelle 4.13).

Die Personalisierung funktioniert. Über alle 18 Sitzungen sinkt der Testfehler von 30,32 % (Konstant-Baseline) auf 19,79 % mit dem Basismodell `norm_subject` und auf 19,59 % mit dem Basismodell `norm_random`. Das ist eine Verringerung um 35 % bzw. 36 % (Wilcoxon-Vorzeichen-Rang-Test $p = 1{,}5 \cdot 10^{-5}$ für beide; gepaarter t-Test $p < 10^{-6}$).

Tabelle 4.13.: Personalisierung der 18 Sitzungen mit zwei Basismodellen (ursprüngliche Labels, zufälliger 70/15/15-Split, Seed 42). Basis: Testfehler des unveränderten Basismodells (Epoche 0); Val.: bester Validierungsfehler; Test: Testfehler in der besten Validierungsepoche; Ep.: beste Epoche; t: Dauer eines Laufs in Sekunden (Abschnitt 4.4.3); Konst.: Konstant-Baseline (Mittelwert der Trainingsziele, auf denselben Testbildern). Fehler in % (Gleichung (4.1)). Gruppenzeilen: Mittelwert und 98 %-KI über die Sitzungen der Gruppe.

		Basismodell `norm_subject`					Basismodell `norm_random`					
Sitzung	Gen.	Basis	Val.	Test	Ep.	t (s)	Basis	Val.	Test	Ep.	t (s)	Konst.
01	1	29,19	14,82	15,83	22	2,3	29,49	11,70	12,97	47	4,2	29,45
02	1	28,48	10,08	10,48	30	2,2	28,42	10,04	10,43	44	3,2	28,49
03	1	29,61	15,26	17,20	33	2,5	30,13	15,41	17,35	36	2,6	29,76
04	1	29,98	16,88	17,64	17	2,1	29,38	15,23	16,16	31	2,7	29,17
05	2	32,92	25,43	17,64	26	1,1	32,49	31,93	32,77	2	0,2	32,72
06	2	29,29	11,31	16,67	54	1,8	28,89	12,09	16,55	55	2,0	27,02
07	2	32,18	24,84	30,99	16	0,7	32,43	25,02	31,62	18	0,6	32,28
08	2	25,82	23,65	23,30	24	0,8	26,32	23,38	22,31	23	0,6	27,00
09	2	26,83	23,55	28,64	4	0,2	26,21	15,19	16,87	47	1,9	28,17
19	4	30,42	13,38	17,76	33	2,8	30,04	12,13	16,85	47	4,4	30,19
20	4	33,14	16,06	14,64	23	1,9	33,01	14,58	12,93	35	2,7	33,49
21	4	30,18	15,03	12,51	39	4,6	30,46	15,80	13,20	23	2,6	29,97
22	4	31,17	13,54	13,02	22	2,2	30,87	12,53	12,32	37	3,1	30,32
23	4	33,15	23,48	23,65	10	1,4	32,59	24,02	24,05	10	2,0	32,57
24	4	33,09	28,77	26,97	7	0,9	32,84	29,53	27,42	6	1,0	33,01
25	4	32,19	26,98	24,61	19	2,4	33,00	26,74	24,13	26	2,8	31,54
26	4	30,60	21,93	18,61	20	2,3	30,39	21,40	17,97	37	4,5	29,48
27	4	31,98	26,05	26,06	19	1,5	32,29	26,27	26,68	19	1,7	31,19
Gen 1 (4)		29,32	14,26	15,29	25,5	2,3	29,35	13,10	14,23	39,5	3,2	29,22
		[27,86; 30,77]	[7,62; 20,89]	[7,80; 22,77]		[1,8; 2,7]	[27,75; 30,96]	[7,07; 19,12]	[7,10; 21,35]		[1,5; 4,8]	[27,99; 30,45]
Gen 2 (5)		29,41	21,76	23,45	24,8	0,9	29,27	21,52	24,02	29,0	1,0	29,44
		[24,14; 34,68]	[11,88; 31,63]	[12,74; 34,16]		[0,0; 1,9][a]	[24,06; 34,47]	[8,20; 34,85]	[10,93; 37,12]		[0,0; 2,5][a]	[24,68; 34,20]
Gen 4 (9)		31,77	20,58	19,76	21,3	2,2	31,72	20,33	19,51	26,7	2,8	31,31
		[30,60; 32,94]	[14,66; 26,50]	[14,26; 25,26]		[1,2; 3,3]	[30,51; 32,93]	[13,88; 26,79]	[13,59; 25,42]		[1,7; 3,9]	[29,91; 32,70]
Alle (18)		**30,57**	**19,50**	**19,79**	**23,2**	**1,9**	**30,51**	**19,05**	**19,59**	**30,2**	**2,4**	**30,32**
		[29,27; 31,87]	[15,87; 23,13]	[16,18; 23,40]		[1,3; 2,5]	[29,21; 31,82]	[14,86; 23,25]	[15,47; 23,71]		[1,6; 3,2]	[29,12; 31,53]

[a] Die untere Grenze des t-Intervalls liegt unter null (wenige Sitzungen, große Streuung) und wurde bei 0 abgeschnitten.

Die unveränderten Basismodelle sind nicht besser als die Konstant-Baseline (30,57 % und 30,51 %; Wilcoxon $p = 0{,}25$ und $p = 0{,}35$). Der ganze Gewinn kommt also aus den wenigen Minuten persönlicher Daten. Die beste Sitzung, S02 (Gen 1, heller Raum, ohne Brille), erreicht 10,5 %. Die neuen Läufe bestätigen die älteren Notebook-Läufe (Mittelwert 19,85 % über dieselben Sitzungen mit `norm_subject`).

Gruppen. Gegenüber der Konstant-Baseline sinkt der Fehler um 48 % / 51 % in Gen 1, um 20 % / 18 % in Gen 2 und um 37 % / 38 % in Gen 4 (`norm_subject` / `norm_random`). In Gen 4 ist die Verbesserung signifikant (Wilcoxon $p = 0{,}004$). Für Gen 1 ($n = 4$) und Gen 2 ($n = 5$) ist der kleinste mögliche p-Wert dieses Tests 0,125 bzw. 0,0625. Diese Gruppen sind also zu klein für ein signifikantes Ergebnis, obwohl der Effekt in Gen 1 groß ist. Die Unterschiede zwischen den Generationen sind nicht signifikant (Kruskal-Wallis $p = 0{,}13$ und $p = 0{,}12$). Man darf sie nicht allein als Wirkung des Aufnahmeablaufs deuten (Abschnitt 4.6).

Wenn die Personalisierung scheitert. In zwei Läufen ist das personalisierte Modell nicht besser als die Konstant-Baseline: S09 mit `norm_subject` (beste Epoche 4, 28,64 % gegenüber 28,17 %) und S05 mit `norm_random` (beste Epoche 2, 32,77 % gegenüber 32,72 %). In beiden Fällen hat sich der Validierungsfehler sehr früh nicht mehr verbessert. Mit dem jeweils anderen Basismodell erreichen dieselben Sitzungen 16,87 % (S09) und 17,64 % (S05). Das Scheitern liegt also nicht an der Sitzung, sondern an einem einzelnen Lauf: Mit nur 20 bis 30 Validierungsbildern (Gen 2) kann das Early Stopping das Training schon nach wenigen Epochen beenden. S07 (dunkler Raum) verbessert sich mit beiden Basismodellen nur wenig (31,0 % / 31,6 % gegenüber 32,3 %). Gutes Licht bleibt also eine Voraussetzung für die Personalisierung mit Webcam.

Basismodell. Beide Basismodelle ergeben dasselbe Ergebnis (19,79 % gegenüber 19,59 %; Wilcoxon $p = 0{,}20$). Das passt zum Hauptergebnis: Das Basismodell liefert vor allem einen Startpunkt, und die persönlichen Daten bestimmen das Ergebnis. In den folgenden Experimenten verwende ich das Basismodell `norm_subject`, weil es ohne Information über die Testpersonen trainiert wurde.

Validierungs- und Testfehler. Im Mittel liegt der Validierungsfehler (19,50 %) nahe am Testfehler (19,79 %). Die größten Abweichungen (5 bis 8 Prozentpunkte, S05 bis S07, S09) gibt es bei den kurzen Gen-2-Aufnahmen. Dort haben Validierung und Test jeweils nur etwa 25 Bilder.

4.4.3. Trainingszeit einer Sitzung

Die Zeit t in Tabelle 4.13 ist die echte Dauer *eines ganzen Personalisierungslaufs* auf den gespeicherten Merkmalen: Auswertung des unveränderten Modells (Epoche 0), alle Epochen bis zum Early Stopping und nach jeder Epoche die Auswertung auf Trainings-, Validierungs- und Testdaten. Sie ist weder eine Zeit pro Epoche noch ein Mittelwert über mehrere Läufe. Sie enthält nicht die einmalige Berechnung der Backbone-Merkmale der Sitzung. Ein Lauf dauert auf einer CPU im Mittel 1,9 s (`norm_subject`) bzw. 2,4 s (`norm_random`). Die Dauer wächst mit der Zahl der Bilder und der Epochen (kurze Gen-2-Aufnahmen: 0,9 s). Ältere Notebook-Läufe, die jedes Bild in jeder Epoche durch den Backbone schickten, brauchten 1 bis 26 Minuten pro Sitzung. Die „Laufzeit“ in den Hyperparametertabellen (Abschnitt 4.1.4) ist dagegen ein Mittelwert über fünf Läufe einer GazeCapture-Einstellung.

4.4.4. Bereinigung der Labels (Gen 4, E1b)

In Gen 4 beginnt jeder Abschnitt mit einer WAIT-Phase von 0,5 s. Das Ziel erscheint an einer neuen Stelle, und der Nutzer muss es erst finden. Diese Bilder (10,0 bis 10,5 % der Labels) können ein falsches Blicklabel haben. Weil sie genau protokolliert sind, kann man bereinigte Labeldateien ohne WAIT-Bilder schreiben (`session_design.py --write-clean`).

Ein einfacher Vergleich wäre aber unfair: Das bereinigte Modell wird ohne WAIT-Bilder getestet, das ursprüngliche Modell dagegen mit allen Bildern, also auch mit den schwer vorhersagbaren WAIT-Bildern. Deshalb werte ich das ursprüngliche Modell zusätzlich nur auf den MOVE-Bildern seiner Testdaten aus (gleiches Modell, WAIT-Bilder aus dem Test entfernt; Tabelle 4.14).

Ergebnis. Die WAIT-Bilder werden sehr schlecht vorhergesagt (37,7 %, schlechter als die Konstant-Baseline). Der Blick ist noch nicht auf dem Ziel, ihre Labels sind also falsch. Entfernt man sie nur aus den *Test*daten, sinkt der Fehler von 19,76 % auf 16,72 %. Entfernt man sie auch aus den *Trainings*daten, sinkt er weiter auf 15,88 %. Dieser Gewinn von 0,84 Prozentpunkten ist nicht signifikant (Wilcoxon $p = 0{,}43$). Er kommt von zwei Sitzungen mit unruhigem Hintergrund (S23, S25, je etwa −3,7 Punkte). In den ruhigen Sitzungen ist der Unterschied kleiner als 0,6 Punkte. Fazit: Die WAIT-Bilder sollte man mindestens aus der Auswertung entfernen, weil sie den Fehler der Labels messen und nicht den Fehler des Modells. Sie auch aus dem Training zu entfernen, kostet nichts und hilft in schwierigen Sitzungen; der Effekt ist aber klein.

Tabelle 4.14.: Bereinigung der Labels in Gen 4 (Basismodell `norm_subject`, zufälliger Split, Seed 42). Ursprünglich: mit allen Labels trainiert; Testfehler auf allen Testbildern, nur auf MOVE und nur auf WAIT (n_W = Anzahl der WAIT-Testbilder). Bereinigt: ohne WAIT-Bilder trainiert und getestet. Δ = Bereinigt − Ursprünglich (MOVE).

	Ursprüngliche Labels				Bereinigte Labels		
Sitzung	Alle	MOVE	WAIT	n_W	Test	Konst.	Δ
19	17,76	13,28	41,12	8	13,06	29,07	−0,22
20	14,64	12,24	28,38	8	12,77	30,71	+0,53
21	12,51	9,77	27,59	10	9,79	29,23	+0,01
22	13,02	10,21	27,64	8	9,96	31,71	−0,25
23	23,65	20,95	41,84	10	17,18	30,92	−3,77
24	26,97	24,96	41,93	10	23,99	31,64	−0,97
25	24,61	21,63	44,36	10	17,94	30,49	−3,70
26	18,61	14,34	41,72	10	14,58	30,20	+0,24
27	26,06	23,10	44,80	8	23,69	30,59	+0,60
Mittel	19,76	16,72	37,71	9,1	15,88	30,51	−0,84

Schwankung zwischen Läufen. Zwei ältere Notebook-Läufe mit gleicher Einstellung auf denselben Daten (Gen 4, Seeds nicht fest) unterschieden sich im Mittel um 1,9 und höchstens um 5,5 Prozentpunkte. Die neuen Läufe mit drei festen Seeds (Tabelle 4.15) zeigen eine Standardabweichung von bis zu 3,1 Prozentpunkten zwischen den Seeds. Unterschiede zwischen *einzelnen* Läufen in dieser Größe kann man also nicht deuten. Das erklärt auch die Fälle S05 und S09 oben.

4.4.5. Aufnahmebedingungen (Gen 4)

In Gen 4 kann man Sitzungen vergleichen, die sich nur in einem Faktor unterscheiden. Jeder Wert ist die Differenz zweier einzelner Sitzungen (Testfehler aus Tabelle 4.13, `norm_subject` / `norm_random`, in Prozentpunkten; positiv bedeutet: die erste Bedingung ist schlechter):

- **Brille** (mit − ohne): +5,2/ + 3,6 (S19–S21), +1,6/ + 0,6 (S20–S22), −3,3/ − 3,4 (S23–S24), +6,0/ + 6,2 (S25–S26);
- **Dauer** (90 s − 120 s): −3,1/ − 3,9 (S20–S19), +0,5/ − 0,9 (S22–S21);
- **Hintergrund** (unruhig − ruhig): +5,9/ + 7,2 (S23–S19), +14,5/ + 14,2 (S24–S21);
- **Betriebssystem** (Windows − Ubuntu): +1,0/ + 0,1 (S25–S23), −8,4/ − 9,4 (S26–S24).

Der einzige einheitliche und große Effekt ist der **Hintergrund**: Ein unruhiger Hintergrund erhöht den Fehler mit beiden Basismodellen um 6 bis 15 Prozentpunkte. Eine Brille erhöht den Fehler meistens (drei von vier Paaren). Die Dauer (90 vs. 120 s) zeigt keinen klaren Effekt, und der Vergleich der Betriebssysteme ist widersprüchlich. Jede Bedingung kommt

nur in einzelnen Sitzungen vor. Diese Werte beschreiben also nur, sie sind nicht statistisch getestet.

4.5. Strategien zum Aufteilen der Daten (E2)

Der zufällige Split aus Abschnitt 4.4 hat eine Schwäche: Benachbarte Bilder eines Videos sind fast gleich (6 Labels pro Sekunde). Die Testdaten enthalten also fast exakte Kopien von Trainingsbildern. Das ist ein Datenleck (Data Leakage), und es macht den zufälligen Split zu optimistisch. Der folgende Vergleich misst, wie groß dieser Effekt ist.

Sitzungen. Ich verwende die Sitzungen 21 und 22. Ich wähle sie nach einer festen Regel: Sie sind die einzigen zwei Sitzungen, die sich nur in einem Faktor unterscheiden (Dauer 120 s vs. 90 s). Sonst wurden sie unter den besten Bedingungen aufgenommen (Gen 4, ohne Brille, ruhiger Hintergrund, Ubuntu, gleicher Tag). Sie eignen sich daher für den Transfer zwischen zwei Videos, und beide haben genug Bilder (720 und 540) für zeitliche Blöcke.

Strategien.

Zufällig (`--split random`): 70 % / 15 % / 15 % der Bilder werden zufällig gezogen (Referenz; enthält fast gleiche Bilder in Training und Test).

Sequenziell (`--split sequential`): Die Bilder bleiben in der Reihenfolge der Aufnahme. Die ersten 70 % sind Training, die nächsten 15 % Validierung und die letzten 15 % Test. Am Anfang des Validierungs- und des Testblocks wird jeweils eine Sekunde (6 Labels) weggelassen, damit keine Nachbarbilder auf beiden Seiten einer Grenze liegen. Das entspricht der echten Nutzung: Der Nutzer kalibriert zuerst und benutzt den Tracker danach.

Zwei Videos (`--split two_videos`): 80 % der Bilder der ersten Sitzung sind Training, die übrigen 20 % (zufällig) Validierung. Die zweite Sitzung dient *nur* als Test (alle Bilder). Ich teste beide Richtungen (21→22 und 22→21). So sieht man, ob eine Kalibrierung auch für eine neue Aufnahme derselben Person gilt.

Fairer Vergleich. Alle Strategien verwenden dasselbe Basismodell (`norm_subject`), dieselben Trainingseinstellungen, Early Stopping auf ihren eigenen Validierungsdaten und dieselben drei Seeds (42, 43, 44). Für jeden Lauf gebe ich die Konstant-Baseline (Mittelwert der *Trainings*ziele, ausgewertet auf den Testbildern dieses Laufs) und den Fehler des unveränderten Basismodells auf denselben Testbildern an. So kann man die verschiedenen Testdaten vergleichen. Kein Testbild wird für Training oder Modellauswahl verwendet.

Tabelle 4.15.: Strategien zum Aufteilen der Daten (Sitzungen 21 und 22, Basismodell `norm_subject`, ursprüngliche Labels). Mittelwert über drei Seeds; SD: Standardabweichung des Testfehlers über die Seeds; Basis: unverändertes Basismodell; Red.: Verringerung des Testfehlers gegenüber der Konstant-Baseline; n: Trainings- / Validierungs- / Testbilder; t: Dauer eines Laufs (s).

Strategie	Training → Test	Test (%)	SD	Val. (%)	Basis (%)	Konst. (%)	Red. (%)	n	t (s)
Zufällig	21 → 21	14,40	3,13	14,15	31,52	31,25	53,9	503 / 108 / 109	3,9
	22 → 22	13,30	0,38	15,98	30,97	30,68	56,7	378 / 81 / 81	3,1
Sequenziell	21 → 21	14,47	1,22	19,09	27,62	27,43	47,2	503 / 102 / 103	2,0
	22 → 22	16,63	0,14	14,41	28,83	29,91	44,4	378 / 75 / 75	2,4
Zwei Videos	21 → 22	16,79	0,13	15,67	31,05	32,17	47,8	576 / 144 / 540	5,0
	22 → 21	17,70	0,42	14,33	32,03	33,13	46,6	432 / 108 / 720	3,2
Zufällig (Mittel)		13,85			31,25	30,97	55,3		
Sequenziell (Mittel)		15,55			28,22	28,67	45,8		
Zwei Videos (Mittel)		17,24			31,54	32,65	47,2		

Ergebnis. Die erwartete Reihenfolge stimmt: zufällig (13,85 %) < sequenziell (15,55 %) < zwei Videos (17,24 %). Mit 6 Läufen pro Strategie (zwei Sitzungen × drei Seeds) ist nur der Unterschied zwischen sequenziell und zwei Videos signifikant (Mann-Whitney U, $p = 0{,}009$). Zufällig vs. sequenziell ($p = 0{,}09$) und zufällig vs. zwei Videos ($p = 0{,}06$) sind es nicht. Weil die Konstant-Baselines der Testdaten verschieden sind, ist die relative Verringerung das fairere Maß: 55 % beim zufälligen Split, aber nur 46 % bis 47 % bei den realistischen Strategien. Der zufällige Split überschätzt den Nutzen der Personalisierung also um etwa 8 bis 9 Prozentpunkte der Verringerung.

Deutung.

- Der zufällige Split ist zu optimistisch, bei diesen beiden guten Sitzungen aber nur mäßig (1,7 Prozentpunkte). Bei schwierigen Sitzungen ist der Effekt viel größer (Abschnitt 4.6.2).
- Eine Kalibrierung funktioniert auch für eine zweite Aufnahme derselben Person unter denselben Bedingungen: Auf einem nie gesehenen Video erreicht das Modell 17 % bis 18 % bei 47 % Verringerung. Der Transfer kostet gegenüber dem zufälligen Split etwa 3 Prozentpunkte.
- Die Ergebnisse mit zwei Videos schwanken am wenigsten zwischen den Seeds (SD unter 0,5), weil die Testdaten groß sind (540 bis 720 Bilder). Der zufällige Split schwankt am meisten (SD 3,1 bei S21), weil seine 81 bis 109 Testbilder vom Seed abhängen.
- Beim sequenziellen Split sagt der Validierungsfehler den Testfehler schlecht voraus (S21: 19,1 % Validierung vs. 14,5 % Test). Validierungs- und Testblock zeigen verschiedene Bereiche des Bildschirms.

Für einen Eye-Tracker, der einmal kalibriert und danach benutzt wird, sind die Werte des sequenziellen Splits und der zwei Videos die realistische Schätzung.

4.6. Gestaltung der Kalibriersitzung

Die Gestaltung der Sitzung bestimmt, welche Bildschirmbereiche die Kalibrierdaten abdecken. Weil die Fehler an Rändern und Ecken am größten sind (Abschnitt 4.3), soll die Gestaltung diese Bereiche gut abdecken. Ich habe drei Varianten aufgenommen:

- **Standardvideo** (Gen 1, `base_Code.py`): Das Ziel bewegt sich gleichmäßig und ohne Sprünge über den Bildschirm;
- **Zufällige Sprünge** (Gen 2): Das Ziel springt alle 1,4 bis 2,7 s an einen zufälligen neuen Startpunkt;
- **Ecken-gewichtete Sprünge** (Gen 4, `base_Code_rand.py`): 5-s-Abschnitte mit 0,5 s WAIT an einem neuen Startpunkt und 4,5 s MOVE; 90 % der Startpunkte liegen in den vier Eckbereichen (je 10 % × 10 % des Bildschirms).

4.6.1. Abdeckung des Bildschirms und Übergangsbilder

Tabelle 4.16 wird direkt aus den Label- und Positionsdateien der 18 Sitzungen berechnet (`session_design.py`); dafür ist kein Training nötig. Die Anteile der Regionen nutzen die Einteilung aus Abschnitt 4.3.1 ($r = 0{,}15$). Übergangsbilder sind Labels, die aufgenommen wurden, während der Blick noch zum neuen Ziel wandert: in Gen 4 die protokollierte WAIT-Phase, in Gen 2 die Labels bis 0,5 s nach einem Sprung (geschätzt, ohne Phasenprotokoll), in Gen 1 keine.

Tabelle 4.16.: Abdeckung des Bildschirms und Übergangsbilder pro Gestaltung (Mittelwert über die Sitzungen jeder Generation). Zellen: Anteil der 100 Zellen eines 10 × 10-Rasters mit mindestens einem Label; Abst.: mittlerer Abstand der Ziele von der Bildschirmmitte (normierte Einheiten).

Gestaltung	Labels	Länge	Sprünge	Übergang	Anteil der Labels (%)			Zellen	Abst.
		(s)		(%)	Mitte	Ränder	Ecken	(%)	
Standard (Gen 1, 4 Sitz.)	493	82	0	0	7,1	25,0	23,0	97	0,383
Zufällige Sprünge (Gen 2, 5 Sitz.)	179	30	16	28,5[a]	6,3	23,0	26,3	57	0,384
Ecken-gewichtet (Gen 4, 9 Sitz.)	644	107	20	10,1	6,1	19,5	36,1	91	0,420

[a] Geschätzt: Labels bis 0,5 s nach einem Sprung.

Beobachtungen.

- **Die Gewichtung der Ecken wirkt wie geplant:** Gen 4 hat 36 % seiner Labels in der Nähe der Ecken, Gen 1 nur 23 % und Gen 2 26 %. Gen 4 hat auch den größten mittleren Abstand von der Mitte.
- **Eine gleichmäßige Bewegung deckt den Bildschirm am besten ab:** Gen 1 erreicht 97 % der Rasterzellen. Die kurzen Gen-2-Aufnahmen (30 s) decken nur 57 % ab.
- **Übergangsbilder:** Mit Sprüngen alle 1,4 bis 2,7 Sekunden und ohne Phasenprotokoll fällt bis zu ein Drittel der Gen-2-Labels in einen Übergang. Gen 4 hat 10 %; diese sind genau protokolliert und können entfernt werden (Abschnitt 4.4.4).

4.6.2. Ergebnis der Personalisierung (E3)

Der zufällige Split bevorzugt Gestaltungen mit langsamer, gleichmäßiger Bewegung, weil fast gleiche Nachbarbilder in Training und Test landen. Deshalb habe ich alle 18 Sitzungen zusätzlich mit dem sequenziellen Split trainiert (Abschnitt 4.5; Basismodell `norm_subject`, Seed 42). Tabelle 4.17 vergleicht beide Splits pro Gestaltung. Tabelle 4.18 zeigt die Fehler pro Bildschirmregion, berechnet aus den gespeicherten Testvorhersagen.

Tabelle 4.17.: Testfehler pro Gestaltung mit zufälligem und sequenziellem Split (Mittelwert über die Sitzungen; Basismodell `norm_subject`). Konst.: Konstant-Baseline der jeweiligen Testbilder; Ep. 0: Anzahl der Sitzungen mit bester Epoche 0 (keine Verbesserung gegenüber dem Basismodell); n_{Test}: mittlere Anzahl der Testbilder.

	Zufälliger Split		Sequenzieller Split			
Gestaltung	Test	Konst.	Test	Konst.	Ep. 0	n_{Test}
Standard (Gen 1)	15,29	29,22	21,52	28,70	1 / 4	69
Zufällige Sprünge (Gen 2)	23,45	29,44	32,80	34,39	2 / 5	22
Ecken-gewichtet (Gen 4)	19,76	31,31	28,07	31,82	5 / 9	91
Alle (18)	19,79	30,32	27,93	31,84[a]	8 / 18	67

[a] Mittelwert der Sitzungswerte des sequenziellen Splits.

Tabelle 4.18.: Testfehler pro Bildschirmregion (Abschnitt 4.3.1, $r = 0{,}15$), Mittelwert über die Sitzungen einer Gestaltung; in Klammern die Gesamtzahl der Testbilder in der Region. Basismodell `norm_subject`.

	Zufälliger Split				Sequenzieller Split			
Gestaltung	Alle	Mitte	Ränder	Ecken	Alle	Mitte	Ränder	Ecken
Standard (Gen 1)	15,3	17,7 (20)	13,5 (67)	16,7 (83)	21,5	11,9 (25)	20,9 (64)	28,9 (52)
Zufällige Sprünge (Gen 2)	23,4	13,7 (12)	21,6 (34)	26,1 (42)	32,8	6,6 (2)[a]	29,8 (15)	38,6 (45)
Ecken-gewichtet (Gen 4)	19,8	11,5 (45)	14,9 (175)	26,5 (299)	28,1	9,0 (60)	23,9 (136)	37,1 (289)
Alle (18)	19,8	13,5 (77)	16,4 (276)	24,2 (424)	27,9	9,7 (87)	24,3 (215)	36,1 (386)

[a] Nur eine Sitzung mit zwei Bildern in der Mitte; nicht aussagekräftig.

Sequenzieller Split. Ohne fast gleiche Bilder in Training und Test steigt der Fehler bei jeder Gestaltung, im Mittel von 19,8 % auf 27,9 % (Wilcoxon $p = 4 \cdot 10^{-5}$). Das personalisierte Modell ist immer noch besser als die Konstant-Baseline (Wilcoxon $p = 0{,}002$), aber die Verringerung schrumpft von 35 % auf etwa 12 %. In 8 von 18 Sitzungen (02, 05, 08, 23–27) ist die beste Epoche 0: Training auf den ersten 70 % der Aufnahme verbessert den Fehler auf dem Validierungsblock nicht. Das angegebene Ergebnis ist dann das unveränderte Basismodell. Das passiert vor allem in Gen 4 mit unruhigem Hintergrund (23–27) und bei den kurzen Gen-2-Aufnahmen, deren Validierungs- und Testblöcke nur 17 bis 27 Bilder haben. Der zufällige Split aus Abschnitt 4.4 überschätzt den Nutzen der Personalisierung bei schwierigen Sitzungen also stark.

Gestaltungen. Bei beiden Splits hat das Standardvideo (Gen 1) den kleinsten und die zufälligen Sprünge (Gen 2) den größten Fehler. Auch mit dem sequenziellen Split sind die Unterschiede nicht signifikant (Kruskal-Wallis $p = 0{,}06$). Die Reihenfolge lässt sich durch die Abdeckung erklären (Tabelle 4.16): Gen 1 deckt in jedem Teil der Aufnahme 97 % des Bildschirms ab. Bei den Sprung-Gestaltungen deckt ein zeitlicher Block nur die Ziele ab, die in dieser Zeit gezeigt wurden. Die kurzen Gen-2-Aufnahmen leiden am meisten darunter. Die Gestaltungen sind außerdem mit Licht, Hintergrund und Brille vermischt. Die Reihenfolge ist also eine Tendenz, kein Beweis.

Regionen. Die Ecken sind bei jeder Gestaltung und bei beiden Splits am schwierigsten (24 % zufällig, 36 % sequenziell), die Mitte am leichtesten (10 bis 14 %). Die Eckengewichtete Gestaltung senkt den Eckfehler *nicht*: Gen 4 hat 26,5 % / 37,1 % in den Ecken, das Standardvideo 16,7 % / 28,9 %. Mehr Ecklabels allein helfen also nicht, wenn der Blick in den Ecken schwerer zu schätzen ist (Kopfdrehung, halb geschlossene Augen, Lage der Kamera) und wenn die Ecklabels unter schlechteren Bedingungen aufgenommen wurden (Gen-4-Sitzungen 23–27). Außerdem muss die Sigmoid-Ausgabe in den äußersten Ecken nahe 0 oder 1 liegen.

Folgerung für die Gestaltung. Mit den vorhandenen Daten liefert für eine kurze Kalibrierung eine gleichmäßige Zielbewegung über den ganzen Bildschirm das beste Ergebnis. Sprünge erzeugen Übergangsbilder (Gen 2: bis zu 28 % der Labels). Wenn man Sprünge verwendet, muss man sie protokollieren, damit man diese Bilder entfernen kann. Die Gewichtung der Ecken erhöht den Anteil der Ecklabels wie geplant, senkt aber nicht den Eckfehler.

4.7. Diskussion der Ergebnisse

Hauptergebnis. Auf GazeCapture erreichen alle Einstellungen der Hyperparameterstudie praktisch nur den Fehler eines konstanten Schätzers auf denselben Testdaten: Die besten Einstellungen mit „nur FC“ liegen zwischen 28,32 % und 28,39 %, die Konstant-Baseline bei 28,46 %. Die räumliche Analyse bestätigt, dass die Vorhersagen in der Bildschirmmitte liegen. Die Hyperparameterstudie trennt die Einstellungen daher vor allem nach ihrer *Stabilität*: Optionen, die das Training unruhiger machen (Gauß, ReLU, keine Aktivierung, mehr trainierbare Schichten, sensitiver Verlust), sind schlechter. Aber keine Option verlässt deutlich die Mitte. Die Entscheidungen in Abschnitt 4.2 beruhen daher auf Stabilität, Lernverlauf, Aufwand und methodischen Gründen, wie in jedem Unterabschnitt beschrieben. Einzelne Notebook-Läufe auf `norm_random` (Abbildungen 4.1b und 4.2) erreichten etwa 23,6 % gegenüber 26,8 % für die Konstant-Baseline. Ein kleiner Lerneffekt existiert also, ist aber im Mittel über fünf Läufe nicht sichtbar. Die Basismodelle bestätigen das: Ihr Fehler auf den persönlichen Sitzungen (30,6 %) entspricht der Konstant-Baseline.

Mögliche Ursachen. Zwei Erklärungen kann das neue Protokoll bereits ausschließen: Die Trainingsdaten enthalten jetzt 179 statt fünf Personen, und die Testdaten werden nicht mehr für die Modellauswahl verwendet. Trotzdem bleiben die Vorhersagen in der Mitte. Die wahrscheinlichsten Ursachen sind: (i) Ich verwende ganze Kamerabilder. Die Augen sind darin nur ein kleiner Teil des Bildes. Die eingefrorenen ImageNet-Merkmale enthalten daher wenig Blickinformation. (ii) 1000 Bilder von 179 Personen sind nur etwa sechs Bilder pro Person. Das ist zu wenig, um eine Abbildung für alle Personen zu lernen. Mehr trainierbare Schichten mit so wenigen Daten machen das Training nur instabil (Abschnitt 4.2.3). Gesichts- oder Augenausschnitte als Eingabe und mehr Trainingsdaten sind die nächsten Schritte, die man prüfen sollte.

Personalisierung. Mit demselben eingefrorenen Backbone senkt ein Kopf, der auf einigen hundert Bildern einer Person trainiert wurde, den Fehler gegenüber der Konstant-Baseline um etwa ein Drittel (zufälliger Split). Das dauert auf einer CPU etwa zwei Sekunden. Die Wahl des Basismodells spielt keine Rolle. Das unterstützt die zentrale Idee dieser Arbeit: die dateneffiziente Personalisierung eines vortrainierten Modells. Zwei Einschränkungen sind wichtig: (i) Der zufällige Split ist zu optimistisch. Mit einem zeitlichen Split sinkt die Verringerung im Mittel auf etwa 12 % und in schwierigen Sitzungen auf null. Gute Sitzungen behalten aber den größten Teil des Gewinns, und eine Kalibrierung funktioniert sogar für eine zweite Aufnahme (47 % Verringerung, Abschnitt 4.5). (ii) Die Aufnahmebedingungen (Hintergrund, Licht, Brille) beeinflussen das Ergebnis stärker als die Gestaltung der Sitzung.

Änderungen am Code. Die Fehlermetrik wurde korrigiert. Baselines werden auf derselben Teilmenge wie das Modell berechnet. `exp.py` hat neue Optionen für zufällige Teilmengen (`--sample-seed`) und eine personendisjunkte Validierungsmenge (`--val-size`). Vollständiges Fine-Tuning (`fc_resnet`) kann man mit `--include-resnet` einschließen. Die Personalisierungsläufe sind reproduzierbar (feste Seeds, eingefrorene BatchNorm, Merkmals-Cache, gespeicherte Vorhersagen, `personal_exp.py`).

4.8. Zusammenfassung der Ergebnisse

Tabelle 4.19 fasst alle Experimente zusammen. „Signifikant“ bedeutet $p < 0{,}05$ im Test, der im jeweiligen Abschnitt genannt ist. Alle Fehler sind Diagonalfehler (Gleichung (4.1)) in %. MAE und MSE wurden in diesen Experimenten nicht protokolliert. Der Diagonalfehler (ein skalierter euklidischer RMSE) ist daher das einzige Genauigkeitsmaß, das für alle Experimente vorhanden und vergleichbar ist.

Tabelle 4.19.: Zusammenfassung der Ergebnisse und der gewählten Einstellung. GazeCapture: `norm_subject`, 1000/200 zufällige Bilder, fünf Läufe, Konstant-Baseline 28,46 %; Zeit = mittlere Dauer eines Trainingslaufs in Minuten (Kopf auf gespeicherten Merkmalen). Personalisierung: Mittelwert über die Sitzungen, Zeit = ein Lauf pro Sitzung in Sekunden (Abschnitt 4.4.3).

Experiment	Gewählt	Fehler (%)	Zeit	Ergebnis und Begründung
Baselines	—	28,46	—	Konstant-Baseline auf denselben Testdaten; Zufalls-Baseline 40,36 (Abschnitt 4.1.6)
Zieldarstellung	normiert $[0,1]^2$	—	—	unabhängig vom Gerät; Metrik sinnvoll definiert (Abschnitt 4.1.5)
Batchgröße	32	28,39	0,019 min	kein signifikanter Unterschied (Abschnitt 4.2.1)
Lernrate	10^{-4}	28,37	0,020 min	kein signifikanter Unterschied; bester Lernverlauf (Abschnitt 4.2.2)
Trainierbare Schichten	nur FC	28,36	0,015 min	kleinster und stabilster Fehler; FC + Layer 4: 9,2 min, voll: 15,4 min (Abschnitt 4.2.3)
Aktivierung	Sigmoid	**28,36**	0,022 min	**signifikant** besser als Gauß, ReLU, Keine (Abschnitt 4.2.4)
Verlustfunktion	L1	**28,32**	0,011 min	**signifikant** besser als der sensitive Verlust (Abschnitt 4.3.4)
Datenmenge	1000/200	28,39	0,018 min	größere Menge nur wegen anderer Testdaten besser (Abschnitt 4.2.5)
Split	`norm_subject`	28,39	0,020 min	einziger Split, der unbekannte Personen misst (Abschnitt 4.2.6)
Personalisierung (E1)	nur FC, 18 Sitzungen	**19,79**	1,9 s	**signifikant** besser als Konstant-Baseline (30,32) und Basismodell (30,57), −35 % (Abschnitt 4.4.2)
Basismodell (E0/E1)	`norm_subject`	19,79	1,9 s	gleich wie `norm_random` (19,59; 2,4 s; $p = 0{,}20$); gewählt, weil es keine Information über die Testpersonen enthält
Bereinigung (E1b)	WAIT-Bilder entfernen	15,88	—	vs. 16,72 auf denselben MOVE-Testbildern; −0,84, nicht signifikant; WAIT-Bilder selbst 37,7 (Abschnitt 4.4.4)
Aufteilung der Daten (E2)	sequenziell / zwei Videos	15,55 / 17,24	2–5 s	zufällig 13,85 ist zu optimistisch; zwei Videos signifikant schlechter als sequenziell; Kalibrierung funktioniert auch für ein neues Video (−47 %) (Abschnitt 4.5)
Gestaltung der Sitzung (E3)	Standardvideo	15,29 / 21,52	—	zufälliger / sequenzieller Split; zufällige Sprünge 23,45 / 32,80, Ecken-gewichtet 19,76 / 28,07; nicht signifikant und vermischt; Ecken-Gewichtung senkt den Eckfehler nicht (Abschnitt 4.6.2)

5. Diskussion

5.1. Antworten auf die Forschungsfragen

FF1: Wie stark hilft eine kurze Personalisierung? Das allgemeine Basismodell ist auf den eigenen Sitzungen nicht besser als die Konstant-Baseline (30,57 % gegenüber 30,32 %). Nach der Personalisierung sinkt der Fehler mit zufälligem Split auf 19,79 %, also um etwa ein Drittel (Abschnitt 4.4.2). Das ist hoch signifikant. Die beste Sitzung erreicht 10,5 %. Mit einem zeitlichen Split, der der echten Nutzung entspricht, ist der Gewinn kleiner: Der Fehler liegt bei 27,9 %, die Verringerung bei etwa 12 %, und in 8 von 18 Sitzungen gibt es keine Verbesserung (Abschnitt 4.6.2). In guten Sitzungen bleibt der Gewinn aber groß und gilt sogar für eine zweite Aufnahme (47 % Verringerung, Abschnitt 4.5). *Antwort:* Eine kurze Personalisierung hilft deutlich, aber nur unter guten Aufnahmebedingungen. Der zufällige Split überschätzt den Nutzen.

FF2: Welche Trainingseinstellung eignet sich? Auf GazeCapture unterscheiden sich die meisten Einstellungen nicht signifikant, weil alle Modelle nahe der Konstant-Baseline bleiben (Abschnitt 4.2). Klar ist: Sigmoid ist die beste Aktivierung, und L1 ist besser als der nach Position gewichtete Verlust. Nur den Kopf zu trainieren („nur FC“) ist genauso gut wie mehr Schichten, aber stabiler und viel schneller. *Antwort:* Eingefrorener Backbone, MLP-Kopf mit Sigmoid, L1-Verlust, Lernrate 10^{-4} und Batchgröße 32. Die Wahl beruht vor allem auf Stabilität und Aufwand.

FF3: Welche Rolle spielen Datenaufteilung und Kalibrierung? Die Aufteilung ist sehr wichtig. Der zufällige Split ist zu optimistisch, weil fast gleiche Nachbarbilder in Training und Test landen. Die gleichmäßige Bewegung des Ziels (Gen 1) deckt den Bildschirm am besten ab und hatte den kleinsten Fehler. Sprünge erzeugen Bilder mit falschen Labels. Diese WAIT-Bilder zu entfernen, verbessert das Ergebnis leicht (0,84 Prozentpunkte, nicht signifikant). Mehr Labels in den Ecken senken den Eckfehler nicht. *Antwort:* Eine kurze, gleichmäßige Bewegung über den ganzen Bildschirm ist am besten. Sprünge sollte man protokollieren, damit man Übergangsbilder entfernen kann. Bewerten sollte man immer mit einem zeitlichen Split.

FF4: Welchen Einfluss haben die Aufnahmebedingungen? Der stärkste Faktor ist der Hintergrund: Ein unruhiger Hintergrund erhöht den Fehler um 6 bis 15 Prozentpunkte. Eine Brille erhöht ihn meistens, und in dunklen Räumen hilft die Personalisierung kaum. Dauer und Betriebssystem zeigen keinen klaren Effekt (Abschnitt 4.4.5). *Antwort:* Ruhiger Hintergrund und gutes Licht sind wichtiger als die Gestaltung der Kalibrierung. Diese Aussagen beruhen aber auf einzelnen Sitzungen.

5.2. Einordnung in den Stand der Technik

Einige Ergebnisse passen gut zur Literatur:

- **Wenige Parameter reichen.** Wie bei der Offset-Kalibrierung [27], bei SPAZE [28] und bei PIGaze [39] genügt hier ein kleiner, angepasster Kopf auf einem eingefrorenen Backbone. Mehr trainierbare Schichten machen das Training nur instabiler. Das passt zu Kumar et al. [22].
- **Personalisierung ist der wichtigste Faktor.** Wie bei Valliappan et al. [7] kommt fast der ganze Gewinn aus wenigen Sekunden persönlicher Daten.
- **Das Evaluationsprotokoll entscheidet.** Ähnlich wie beim Unterschied zwischen `norm_random` und `norm_subject` überschätzen zufällige Splits die Leistung (Abschnitt 2.7.1).
- **Falsche Labels haben einen kleinen Effekt.** Auch DMRF [41] senkt den Fehler nur wenig (3,92° → 3,85°). Hier bringt das Entfernen der WAIT-Bilder 0,84 Prozentpunkte.

Es gibt aber auch einen wichtigen Unterschied: Das allgemeine Modell dieser Arbeit ist sehr schwach. Es bleibt bei der Bildschirmmitte, während Verfahren in der Literatur auch ohne Personalisierung einige Grad Genauigkeit erreichen. Der wichtigste Grund ist wahrscheinlich die Eingabe: Die meisten Verfahren nutzen Gesichts- oder Augenausschnitte, diese Arbeit dagegen das ganze Kamerabild. Außerdem nutzt das Basismodell nur 1000 Bilder von 179 Personen. Die absoluten Zahlen sind wegen der unterschiedlichen Metriken ohnehin nicht direkt mit der Literatur vergleichbar (Abschnitt 2.10).

5.3. Grenzen der Arbeit

- **Eine Person:** Alle eigenen Aufnahmen stammen von einer Person. Die Ergebnisse gelten daher nicht automatisch für andere Personen.

- **Wenige Sitzungen und vermischte Bedingungen:** 18 Sitzungen reichen nicht für sichere Aussagen über einzelne Faktoren. Gestaltung, Licht, Hintergrund und Brille ändern sich oft gleichzeitig.
- **Einzelne Läufe:** Pro Sitzung gibt es meist nur einen Lauf (Seed 42). Die Streuung zwischen Seeds kann bis zu 3 Prozentpunkte betragen.
- **Ein Backbone und ganze Bilder:** Untersucht wurde nur ResNet-18 mit ganzen Kamerabildern. DINOv2, CLIP, MAE, UniGaze, Adapter und LoRA wurden nicht getestet, ebenso wenig Gesichts- oder Augenausschnitte in den Experimenten von Kapitel 4.
- **Schwaches Basismodell:** Das Basismodell lernt auf GazeCapture kaum etwas. Ein besseres Basismodell könnte die Personalisierung verändern.
- **Metrik:** Der Diagonalfehler ist nicht direkt mit Winkelfehlern oder Zentimetern aus der Literatur vergleichbar.

6. Fazit und Ausblick

6.1. Fazit

Diese Arbeit hat untersucht, wie gut man ein vortrainiertes Bildverarbeitungsmodell mit einer kurzen Webcam-Kalibrierung an eine Person anpassen kann. Ein auf ImageNet vortrainiertes ResNet-18 blieb eingefroren, und nur ein kleiner Regressionskopf wurde trainiert.

Die wichtigsten Ergebnisse sind:

- Ein allgemeines Modell, das auf GazeCapture trainiert wurde, ist nicht besser als eine Schätzung, die immer die Bildschirmmitte ausgibt.
- Schon wenige Minuten persönlicher Daten senken den Fehler deutlich: von 30,3 % auf 19,8 % der Bildschirmdiagonale (zufälliger Split, 18 Sitzungen). Ein Lauf dauert etwa zwei Sekunden auf einer CPU.
- Mit einem realistischen, zeitlichen Split ist der Gewinn kleiner (etwa 12 %). In guten Sitzungen bleibt er aber groß und gilt auch für eine neue Aufnahme derselben Person.
- Ruhiger Hintergrund und gutes Licht sind wichtiger als die Gestaltung der Kalibrierung. Eine gleichmäßige Bewegung des Ziels über den ganzen Bildschirm war am besten.
- Bilder direkt nach einem Sprung des Ziels haben falsche Labels. Sie zu entfernen hilft wenig, kostet aber nichts.

Insgesamt zeigt die Arbeit: Dateneffiziente Personalisierung funktioniert mit einfachen Mitteln. Die Genauigkeit hängt aber stark von den Aufnahmebedingungen und von einer ehrlichen Auswertung ab.

6.2. Ausblick

Mehrere nächste Schritte liegen nahe:

- **Augen- oder Gesichtsausschnitte als Eingabe.** Im ganzen Kamerabild sind die Augen sehr klein. Ein Ausschnitt sollte dem Modell viel mehr Blickinformation geben.

- **Andere vortrainierte Modelle.** DINOv2, CLIP, MAE oder UniGaze statt ResNet-18, jeweils mit eingefrorenem Backbone und kleinem Kopf.
- **Sparsame Anpassung des Backbones.** Adapter oder LoRA als Mittelweg zwischen reinem Kopftraining und vollständigem Fine-Tuning.
- **Mehr Personen und mehr Sitzungen,** mit Bedingungen, die sich jeweils nur in einem Faktor ändern.
- **Kopfbewegung berücksichtigen,** zum Beispiel mit der Kopfposition als zusätzlichem Merkmal oder mit normalisierten Gesichtsbildern.

Web-Demo. Als praktischen Test habe ich eine Demo im Browser gebaut (Server mit FastAPI, geeignet für Hugging Face Spaces). Der Nutzer folgt etwa 60 Sekunden lang einem Punkt. Danach wird ein Modell personalisiert, und der Blick wird live als Lichtfeld auf dem Bildschirm angezeigt. Am Ende zeigt eine Auswertung, wann welcher Bildschirmbereich angeschaut wurde. Die Demo nutzt bereits einige Ideen aus dem Ausblick: einen Augenausschnitt, räumliche Merkmale des ResNet-18, eine Ridge-Regression und die Kopfposition als Zusatzmerkmal. Ein kurzer Test mit fünf Punkten nach der Kalibrierung misst die Genauigkeit ehrlich. In zwei einzelnen Tests sank dieser Fehler mit der Kopfposition von 23,0 % auf 17,6 %. Diese Werte stammen aus Einzeltests und wurden nicht systematisch ausgewertet. Sie zeigen aber, dass Kopfbewegung im Live-Betrieb eine der größten Fehlerquellen ist.

Literaturverzeichnis

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik und Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren und Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 770–778, 2016.

[3] Yihua Cheng, Haofei Wang, Yiwei Bao und Feng Lu. Appearance-based gaze estimation with deep learning: A review and benchmark. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):7509–7528, 2024. doi: 10.1109/TPAMI.2024.3393571.

[4] Jiahui Liu, Jiannan Chi, Huijie Yang und Xucheng Yin. In the eye of the beholder: A survey of gaze tracking techniques. *Pattern Recognition*, 132:108944, 2022. doi: 10.1016/j.patcog.2022.108944.

[5] Primesh Pathirana, Shashimal Senarath, Dulani Meedeniya und Sampath Jayarathna. Eye gaze estimation: A survey on deep learning-based approaches. *Expert Systems with Applications*, 199:116894, 2022. doi: 10.1016/j.eswa.2022.116894.

[6] Alexandra Papoutsaki, Patsorn Sangkloy, James Laskey, Nediyana Daskalova, Jeff Huang und James Hays. Webgazer: Scalable webcam eye tracking using user interactions. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence (IJCAI)*, S. 3839–3845, 2016.

[7] Nachiappan Valliappan, Na Dai, Ethan Steinberg, Junfeng He, Kantwon Rogers, Venky Ramachandran, Pingmei Xu, Mina Shojaeizadeh, Li Guo, Kai Kohlhoff und Vidhya Navalpakkam. Accelerating eye movement research via accurate and affordable smartphone eye tracking. *Nature Communications*, 11:4553, 2020. doi: 10.1038/s41467-020-18360-5.

[8] Xucong Zhang, Yusuke Sugano, Mario Fritz und Andreas Bulling. Appearance-based gaze estimation in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 4511–4520, 2015.

[9] Xucong Zhang, Yusuke Sugano, Mario Fritz und Andreas Bulling. Mpiigaze: Real-world dataset and deep appearance-based gaze estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(1):162–175, 2019. doi: 10.1109/TPAMI.2017.2778103.

[10] Xucong Zhang, Seonwook Park, Thabo Beeler, Derek Bradley, Siyu Tang und Otmar Hilliges. Eth-xgaze: A large scale dataset for gaze estimation under extreme head pose and gaze variation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. doi: 10.1007/978-3-030-58558-7_22.

[11] Seonwook Park, Emre Aksan, Xucong Zhang und Otmar Hilliges. Towards end-to-end video-based eye-tracking. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. doi: 10.1007/978-3-030-58610-2_44.

[12] Petr Kellnhofer, Adrià Recasens, Simon Stent, Wojciech Matusik und Antonio Torralba. Gaze360: Physically unconstrained gaze estimation in the wild. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, S. 6911–6920, 2019.

[13] Seonwook Park, Shalini De Mello, Pavlo Molchanov, Umar Iqbal, Otmar Hilliges und Jan Kautz. Few-shot adaptive gaze estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, S. 9368–9377, 2019. doi: 10.1109/ICCV.2019.00946.

[14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit und Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[15] Jiawei Qin, Xucong Zhang und Yusuke Sugano. Unigaze: Towards universal gaze estimation via large-scale pre-training. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, S. 5809–5820, 2026.

[16] Vikrant Nagpure, Tanisha Jain, Rongali Sai Bhargav, Ashwin K. Krishna und Kenji Okuma. Learning to adapt: Test-time personalized gaze estimation with weak and self-supervised learning for in-vehicle scenarios. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, S. 1334–1339, 2025. doi: 10.1109/IV64158.2025.11097756.

[17] Jun Wang, Hao Ruan, Liangjian Wen, Yong Dai und Mingjie Wang. Gazeclip: Enhancing gaze estimation through text-guided multimodal learning. *Neurocomputing*, 685:133584, 2026. doi: 10.1016/j.neucom.2026.133584.

[18] Lang Ye, Xinggang Wang, Jingfeng Yao und Wenyu Liu. Transgaze: exploring plain vision transformers for gaze estimation. *Machine Vision and Applications*, 35:128, 2024. doi: 10.1007/s00138-024-01609-0.

[19] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby u. a. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.

[20] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger und Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Band 139 von *Proceedings of Machine Learning Research*, S. 8748–8763, 2021.

[21] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár und Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 16000–16009, 2022.

[22] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma und Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[23] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang und Weizhu Chen. Lora: Low-rank adaptation of large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[24] Xucong Zhang, Yusuke Sugano und Andreas Bulling. Revisiting data normalization for appearance-based gaze estimation. In *Proceedings of the ACM Symposium on Eye Tracking Research & Applications (ETRA)*, 2018.

[25] Pengwei Yin, Guanzhong Zeng, Jingjing Wang und Di Xie. Clip-gaze: Towards general gaze estimation via visual-linguistic model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Band 38, S. 6729–6737, 2024. doi: 10.1609/aaai.v38i7.28496.

[26] Junfeng He, Khoi Pham, Nachiappan Valliappan, Pingmei Xu, Chase Roberts, Dmitry Lagun und Vidhya Navalpakkam. On-device few-shot personalization for real-time gaze estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

[27] Zhaokang Chen und Bertram E. Shi. Offset calibration for appearance-based gaze estimation via gaze decomposition. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020.

[28] Erik Lindén, Jonas Sjöstrand und Alexandre Proutière. Learning to personalize in appearance-based gaze tracking. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, S. 1140–1148, 2019. doi: 10.1109/ICCVW.2019.00145.

[29] Gang Liu, Yu Yu, Kenneth A. Funes Mora und Jean-Marc Odobez. A differential approach for gaze estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(3):1092–1098, 2021. doi: 10.1109/TPAMI.2019.2957373.

[30] Xiaoyu Zhao, Yaping Huang, Yi Tian und Mei Tian. Episode-based personalization network for gaze estimation without calibration. *Neurocomputing*, 513:36–45, 2022.

[31] Yu Yu, Gang Liu und Jean-Marc Odobez. Improving few-shot user-specific gaze adaptation via gaze redirection synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[32] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan und Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, Band 97 von *Proceedings of Machine Learning Research*, S. 2790–2799, 2019.

[33] Huan Liu, Julia Qi, Zhenhao Li, Mohammad Hassanpour, Yang Wang, Konstantinos N. Plataniotis und Yuanhao Yu. Test-time personalization with meta prompt for gaze estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Band 38, S. 3621–3629, 2024. doi: 10.1609/aaai.v38i4.28151.

[34] Yiwei Bao und Feng Lu. From feature to gaze: A generalizable replacement of linear layer for gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 1409–1418, 2024. doi: 10.1109/CVPR52733.2024.00140.

[35] Yaoming Wang, Yangzhou Jiang, Jin Li, Bingbing Ni, Wenrui Dai, Chenglin Li, Hongkai Xiong und Teng Li. Contrastive regression for domain adaptation on gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 19376–19385, 2022. doi: 10.1109/CVPR52688.2022.01877.

[36] Yihua Cheng, Yiwei Bao und Feng Lu. Puregaze: Purifying gaze feature for generalizable gaze estimation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.

[37] Mingfang Zhang, Yunfei Liu und Feng Lu. Gazeonce: Real-time multi-person gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 4197–4206, 2022. doi: 10.1109/CVPR52688.2022.00416.

[38] Yong Wu, Yang Wang, Sanqing Qu, Zhijun Li und Guang Chen. Elf-ua: Efficient label-free user adaptation in gaze estimation. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[39] Yafei Wang, Haiheng Nan, Runze Yan, Xueyan Ding, Jianda Wang und Xianping Fu. Pigaze: Personalized in-vehicle gaze estimation with plug-and-play adaptation. *Knowledge-Based Systems*, 348:116336, 2026.

[40] Diederick C. Niehorster, Marcus Nyström, Roy S. Hessels, Jeroen S. Benjamins, Richard Andersson und Ignace T. C. Hooge. The fundamentals of eye tracking, part 7: Determining data quality. *Behavior Research Methods*, 58:183, 2026. doi: 10.3758/s13428-026-03039-4.

[41] Gaoxia Jiang, Fan Lei und Wenjian Wang. A multi-model dynamic filtering of label noise for regression. *Pattern Recognition*, 169:111887, 2026. doi: 10.1016/j.patcog.2025.111887.

[42] Yanming Peng, Shijing Wang, Yaping Huang und Yi Tian. See through the noise: Improving domain generalization in gaze estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, S. 31346–31355, 2026.

A. Anhang

A.1. Befehle zum Wiederholen der Experimente

Alle Befehle werden im Hauptordner des Projekts ausgeführt (siehe auch `README.md`).

Daten vorbereiten.

```
python main.py
```

Baselines und Hyperparameterstudie (GazeCapture).

```
python exp.py --experiment baseline
python exp.py --experiment all --sample-seed 0 --val-size 200
python exp.py --experiment trainable_layers --include-resnet
```

Räumliche Analyse und Abbildungen.

```
python spatial_analysis.py --input results/heatmap/base_loss1.csv \
       --compare results/heatmap/base_loss2.csv --out results/figures
python plot_targets.py --out results/figures
```

Personalisierung (E0–E3).

```
python train_base.py
python personal_exp.py --sessions all --base norm_subject norm_random
python personal_exp.py --sessions gen4 --labels clean
python personal_exp.py --sessions 21 22 --split random --seeds 42 43 44
python personal_exp.py --sessions 21 22 --split sequential --seeds 42 43 44
python personal_exp.py --split two_videos --pairs 21:22,22:21 --seeds 42 43 44
python personal_exp.py --sessions all --split sequential
python personal_exp.py --summary
```

Gestaltung der Sitzungen und bereinigte Labels.

```
python session_design.py
python session_design.py --write-clean
```

Einfacher geht es mit dem Notebook `notebooks/Masterarbeit.ipynb`: Zelle 1 trainiert die Basismodelle (E0), Zelle 3 führt die Personalisierung (E1 bis E3) aus und erstellt alle Tabellen.

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich diese Masterarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, alle Ausführungen, die anderen Schriften wörtlich oder sinngemäß entnommen wurden, kenntlich gemacht sind und die Arbeit in gleicher oder ähnlicher Fassung noch nicht Bestandteil einer Studien- oder Prüfungsleistung war.

Verwendung von KI-Werkzeugen

[**Noch zu schreiben:** Angabe nach den Vorgaben der Hochschule, z. B.: welches Werkzeug, wofür (Programmierung und Fehlersuche, Auswertung, sprachliche Überarbeitung, Übersetzung) und dass die Inhalte geprüft und verantwortet wurden. Mit dem Betreuer abstimmen.]

Ort, Datum

Khaled Rafei